ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ» ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК

Ахундов Алексей Назимович, Жарковский Дмитрий Андреевич

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Генерация персонализированных учебных конспектов по предметам ЕГЭ на основе языковых моделей и пользовательской аналитики

Generation of Personalized Educational Notes for Unified State Exam Subjects Based on Language Models and User Analytics

по направлению подготовки 01.04.02 Прикладная математика и информатика
образовательная программа «Искусственный интеллект»

Студент

_____

А.Н. Ахундов

Студент

_____

Д.А. Жарковский

Научный руководитель
Кандидат физико-математических наук

_____

Е.О. Кантонистова

Соруководитель
Преподаватель

_____

П.И. Ахтямов

Москва 2026

Содержание

Аннотация **4**

Abstract **5**

Введение **6**

1 Глава 1. Проблематика персонализированного онлайн-обучения и постановка задачи генерации учебных конспектов **7**

1.1 Цифровизация подготовки к ЕГЭ и особенности рынка онлайн-обучения . . . 7

1.2 Научно-прикладные подходы к персонализации обучения 8

1.3 Критический анализ существующих решений и исследований 10

1.4 Постановка задачи ВКР и методы ее решения 11

1.5 Выводы по главе 13

2 Глава 2. Проектирование и реализация сервиса генерации персонализированных учебных конспектов **14**

2.1 Архитектура сервиса персонализированных конспектов 14

2.2 Обоснование выбора методов и инструментальных средств 16

2.3 Структура классов и интерфейсов проекта 18

2.4 Алгоритм формирования персонализированного конспекта 20

2.5 Реализация сервиса и решения, обеспечивающие масштабируемость 23

2.6 Результат решения задачи и личный вклад участников проекта 25

2.7 Выводы по главе 26

3 Глава 3. Реализация подсистемы извлечения и генерации персонализированных конспектов **28**

3.1 Структура предметной базы знаний 28

3.2 Векторное представление и индексирование учебных материалов 30

3.3 Гибридный поиск и многоуровневое ранжирование 31

3.4 Генерация и верификация конспекта 35

3.5 Выводы по главе 37

4 Глава 4. Интеграция с веб-платформой и эксплуатационные характеристики сервиса **38**

4.1 Постановка инженерной задачи интеграции 38

4.2 Контракты взаимодействия с платформой 39
4.3 Поток 1: фоновая диагностика ответов учеников 40
4.4 Поток 2: генерация персонализированного конспекта по запросу 42
4.5 Семантика надёжности и аутентификации 44
4.6 Развёртывание в Kubernetes и среда разработки 45
4.7 Выводы по главе 47

Список литературы **49**

А Пример сгенерированного конспекта **54**

Аннотация

Подготовка к единому государственному экзамену (ЕГЭ) является одной из наиболее востребованных областей онлайн-образования. Существующие образовательные платформы предлагают тренажёры, видеоразборы и рекомендательные системы, однако персонализация в них, как правило, ограничивается выбором курса. При этом теоретический материал остаётся универсальным: ученик, допускающий устойчивые ошибки, вынужден самостоятельно выстраивать объяснение из разрозненных источников. Проблема усугубляется тем, что большие языковые модели (Large Language Models, LLM), способные генерировать связные объяснения, без внешних ограничений подвержены галлюцинациям и не привязаны к конкретной учебной программе. На данный момент в литературе отсутствуют решения, объединяющие пользовательскую аналитику, управляемую генерацию текста и предметную базу знаний для автоматического создания персонализированных учебных конспектов для заданий ЕГЭ. В данной работе разработан сервис для действующей веб-платформы «ЕГЭ Прорыв» [42], который на основе истории ошибок, динамики прогресса и уровня освоения навыков конкретного ученика автоматически формирует индивидуальный учебный конспект. Используя наш подход, нам удалось получить связные персонализированные конспекты, содержащие объяснения ключевых понятий, разбор типичных ошибок и рекомендации, адаптированные к дефицитам знаний обучающегося. Таким образом, полученные результаты могут быть использованы для повышения эффективности подготовки к ЕГЭ на образовательных платформах, а предложенный подход применим к другим предметам и экзаменационным форматам.

Ключевые слова

Персонализированное обучение, генерация учебных конспектов, ЕГЭ, большие языковые модели в образовании, аналитика обучения

Abstract

Preparation for the Unified State Examination is among the most sought-after areas of online education. Existing educational platforms offer drills, video walkthroughs, and recommender systems; however, personalization is usually limited to course selection. Theoretical material remains generic: a student who makes persistent errors must assemble explanations from scattered sources on their own. The problem is compounded by the fact that large language models (LLM), while able to produce coherent explanations, are prone to hallucinations without external constraints and are not tied to a specific curriculum. At present, the literature offers no solutions that combine user analytics, controlled text generation, and a subject knowledge base for the automatic creation of personalized study notes for Unified State Examination tasks. This work presents a service for the live web platform Exambreakout [42], which automatically builds an individual study note from a student's error history, progress dynamics, and skill mastery. Using our approach, we obtained coherent personalized notes containing explanations of key concepts, analyses of typical errors, and recommendations adapted to the learner's knowledge gaps. Thus, the results can improve the effectiveness of Unified State Examination preparation on educational platforms, and the proposed approach extends to other subjects and examination formats.

Keywords

Personalized learning, study note generation, Unified State Examination, large language models in education, learning analytics

Введение

Цифровизация образования и развитие онлайн-платформ подготовки к ЕГЭ привели к существенному росту доступности учебных материалов, тренажёров и инструментов контроля знаний. Вместе с тем современные образовательные сервисы по-прежнему в основном ориентированы на массовый сценарий обучения: они хорошо решают задачи хранения контента, автоматической проверки ответов и подбора упражнений, но значительно хуже адаптируют теоретическое объяснение к индивидуальным дефицитам знаний конкретного обучающегося. В условиях экзаменационной подготовки это особенно критично, поскольку эффективность повторения определяется не только количеством решённых задач, но и тем, насколько быстро ученик получает понятное объяснение своих типичных ошибок и пробелов. Решения на основе LLM генерируют связные и образовательные объяснения, однако без внешних ограничений они подвержены галлюцинациям и не гарантируют соответствия учебной программе и формату экзамена [19, 39, 36]. В существующих исследованиях и прикладных системах недостаточно проработан сценарий, в котором пользовательская аналитика, предметная база знаний и управляемая генерация объединяются в сервис, автоматически формирующий персонализированный учебный конспект для конкретного ученика.

В данной работе основанием для генерации персонализированных учебных конспектов являются данные о пользовательской активности на платформе, результаты решения заданий, история ошибок, показатели прогресса, а также накопленные предметные материалы по заданиям ЕГЭ. Именно эти исходные данные позволяют перейти от универсальной выдачи теории к адаптивному объяснению, учитывающему профиль обучающегося.

Конечной целью нашей работы является разработка сервиса для платформы «ЕГЭ Прорыв»(находящейся в стадии бета-тестирования) [42], автоматически формирующего персонализированные учебные конспекты по математике с учётом истории ошибок, динамики прогресса и уровня освоения навыков конкретного ученика.

Полученные результаты генерации персонализированных конспектов позволяют обучающемуся получить связное и осмысленное объяснение своих типичных ошибок и пробелов в знаниях, что позволяет повысить эффективность подготовки к ЕГЭ, что доказывает преимущества нашего подхода перед существующими решениями.

1 Глава 1. Проблематика персонализированного онлайн-обучения и постановка задачи генерации учебных конспектов

1.1 Цифровизация подготовки к ЕГЭ и особенности рынка онлайн-обучения

Подготовка к ЕГЭ за последние годы стала одной из наиболее заметных областей цифрового образования. На рынке одновременно присутствуют крупные образовательные платформы и специализированные онлайн-школы, предлагающие курсы, тренажеры, видеоразборы и консультации по предметам экзамена. Это подтверждается как распространенностью профильных сервисов, так и разнообразием их продуктовых моделей: отдельные предметные курсы и тренажеры представлены, например, у Яндекса, ЕГЭLand, Умскул и MAXIMUM [25, 41, 44, 45]. Для пользователя такой рынок создает широкий выбор форматов, однако одновременно усиливает конкуренцию по качеству контента, степени персонализации и скорости адаптации материалов под конкретные образовательные цели.

Современные исследования онлайн-образования показывают, что эффективность цифрового обучения определяется не только наличием учебных материалов, но и качеством структуры курса и уровнем интерактивности [14]. Иными словами, простой перенос теории и задач в веб-интерфейс еще не делает обучение по-настоящему эффективным. Особенно остро это проявляется в экзаменационной подготовке, где учащемуся нужен не просто доступ к контенту, а своевременное объяснение именно тех тем, которые вызывают устойчивые ошибки.

Для подготовки к ЕГЭ эта проблема имеет особую специфику. Во-первых, содержание обучения жестко ограничено кодификатором и спецификацией экзамена, поэтому образовательный сервис должен обеспечивать релевантность и нормативную корректность материалов. Во-вторых, темп усвоения тем у разных учеников заметно различается: один обучающийся испытывает трудности с базовыми определениями, другой — с типовыми преобразованиями, третий — с переносом знаний на задания повышенной сложности. В-третьих, массовые онлайн-курсы обычно строятся вокруг усредненной траектории: ученик получает одинаковую теорию, общий набор практики и ограниченную адаптацию, выраженную чаще всего в рекомендации следующего урока или задания.

Таким образом, на рынке онлайн-подготовки сформировалось противоречие. С одной

стороны, цифровые платформы обеспечивают масштабируемость, доступность и богатую учебную аналитику. С другой стороны, они по-прежнему редко предоставляют персонализированный теоретический материал в форме связного учебного конспекта, отражающего историю ошибок, текущий уровень подготовки и учебные цели конкретного ученика. Именно это противоречие и определяет актуальность разработки специализированного сервиса для платформы «ЕГЭ Прорыв» [42]. Платформа уже содержит действующую систему адаптивного обучения на основе графа микромодулей, байесовской оценки компетенций и кривой забывания [43], а также подсистему оптимизации образовательных траекторий с применением машинного обучения [40]. Однако указанные компоненты управляют *выбором* следующего задания, но не *содержанием* теоретического объяснения, что создаёт нишу для отдельного RAG-сервиса генерации персонализированных конспектов.

1.2 Научно-прикладные подходы к персонализации обучения

В научной литературе персонализированное обучение понимается как организация учебного процесса, при которой содержание, темп, форма представления материала и обратная связь адаптируются под особенности конкретного обучающегося [21]. Такой подход противопоставляется модели, в которой один и тот же учебный материал предъявляется всей аудитории без учета различий в исходной подготовке, стиле усвоения и динамике прогресса.

Методологической основой персонализации выступают учебная аналитика (Learning Analytics) и образовательный сбор данных (Educational Data Mining). По определению R. Baker, Educational Data Mining ориентирован на извлечение закономерностей из учебных данных для понимания и оптимизации обучения [4]. В более позднем обзоре C. Romero и S. Ventura подчеркивается, что Learning Analytics и Educational Data Mining сегодня образуют единую исследовательскую область, в которой анализ цифровых следов обучающегося используется для прогнозирования результатов, выявления трудностей и поддержки педагогических решений [35]. Обзор E. T. Khor и Mutthulakshmi K. показывает, что Learning Analytics способствует персонализации как минимум по пяти направлениям: сбор данных о прогрессе, классификация учащихся, построение циклов обратной связи, прогнозирование результатов и предоставление преподавателю или системе интерпретируемых рекомендаций в реальном времени [21].

Отдельное место в персонализации занимает моделирование знаний обучающегося. Классической работой в этой области считается статья A. Corbett и J. Anderson, где была предложена модель отслеживание знаний (Knowledge Tracing) для оценки вероятности усво-

ения навыка по последовательности ответов ученика [9]. Дальнейшее развитие направление получило в работе C. Piech и соавторов о нейросетевом моделировании отслеживания знаний (Deep Knowledge Tracing), где динамика овладения знаниями моделируется с помощью нейросетевых методов [28]. Современный обзор G. Abdelrahman и соавторов показывает, что моделирование обучения (Learner Modeling) и Knowledge Tracing остаются центральными инструментами адаптивных обучающих систем, особенно в задачах прогнозирования ошибок, выбора индивидуальной траектории и адресной поддержки [1].

Новый этап развития персонализации связан с большими языковыми моделями (Large Language Models, LLM). Работы по BERT, GPT и LLaMA показали, что крупные трансформерные модели способны решать широкий круг задач понимания и генерации текста, включая объяснение понятий, переформулирование материала и адаптацию стиля изложения [12, 5, 37]. Для образования это особенно важно, поскольку учебный контент является прежде всего текстовым знанием, которое должно быть представлено в понятной, логичной и методически корректной форме.

При этом использование больших языковых моделей в образовании нельзя сводить к обычному чат-интерфейсу. Обзор E. Kasneci и соавторов показывает, что большие языковые модели открывают значительные возможности для тьюторской поддержки, генерации объяснений и автоматизации обратной связи, однако одновременно создают риски недостоверности, избыточной уверенности модели и педагогически неудачных подсказок [19]. Систематический обзор L. Yan и соавторов дополняет этот вывод практическими и этическими ограничениями: зависимость обучающихся от подсказок ИИ, непрозрачность алгоритмических решений, проблемы приватности и справедливости, а также сложность надежной оценки качества сгенерированного материала [39]. Более свежий систематический обзор Y. Shi и соавторов подтверждает, что LLM уже активно применяются в интеллектуальных обучающих системах, но их положительный эффект оказывается неоднородным и сильно зависит от сценария внедрения, глубины педагогической интеграции и наличия внешнего контроля [36].

Для повышения надежности генерации используются генерации с дополненной выборкой (Retrieval Augmented Generation, RAG). Подход RAG предполагает дополнение генерации внешним контекстом из базы знаний, что снижает вероятность галлюцинаций и делает ответы более привязанными к предметной области [24, 16]. В образовательном контексте, как показывает обзор Y. Ding и соавторов, RAG особенно полезен тогда, когда необходимо связать генерацию с учебной программой, проверенными материалами и профилем ученика [13].

1.3 Критический анализ существующих решений и исследований

Существующие решения в предметной области можно условно разделить на три класса: массовые онлайн-курсы и платформы подготовки к экзаменам, интеллектуальные тьюторские системы на основе аналитики обучения и современные LLM-ориентированные ассистенты.

Первый класс представлен образовательными платформами, ориентированными на массовый сегмент подготовки к ЕГЭ [41, 44, 25]. Их сильными сторонами являются масштабируемость, насыщенная медиатека, наличие тренажеров и готовых учебных траекторий. Однако такие решения обычно адаптируют обучение на уровне выбора курса, темы или набора задач. В результате теория остается относительно универсальной, а ученик с повторяющимися ошибками нередко вынужден самостоятельно собирать для себя “личный конспект” из видео, заметок, чатов и разрозненных комментариев преподавателя.

Второй класс включает системы, основанные на Learner Modeling, Knowledge Tracing и учебной аналитике, такие платформы не представлены на российском рынке, но есть зарубежные аналоги [9, 28, 1, 21]. Эти подходы хорошо решают задачи диагностики: они позволяют определить вероятность усвоения навыка, выявить “узкие места” и спрогнозировать риск неуспешности. Их принципиальное ограничение состоит в том, что они отвечают на вопрос “что именно ученик знает плохо”, но не решают в полной мере задачу “как именно представить этому ученику новый материал”. Иными словами, аналитика эффективно обнаруживает дефицит, но сама по себе не создает качественное персонализированное объяснение.

Третий класс — это LLM-ориентированные образовательные ассистенты. На прикладном уровне к наиболее заметным примерам относится тренажер подготовки к ЕГЭ от Яндекс учебника [45]. Из зарубежных платформ относящихся к этому классу можем отметить Khan Academy и Q-Chat [20, 32] На научном уровне активно развиваются работы по генерации персонализированной обратной связи и рекомендаций с помощью LLM [27, 38]. Достоинство этих решений заключается в естественности взаимодействия и способности адаптировать объяснение к запросу пользователя. Тем не менее большинство таких систем по-прежнему работает в реактивном диалоговом режиме: они хорошо отвечают на вопрос, но хуже формируют целостный учебный текст, который последовательно раскрывает тему, объясняет проблемные места, удерживает необходимую глубину и при этом не выходит за рамки предметного стандарта.

Критический анализ литературы позволяет выделить несколько нерешенных проблем.

1 **Недостаточная персонализация теоретического материала.** Обзоры по Learning

Analytics показывают, что данные о прогрессе ученика могут использоваться для классификации, прогнозирования и формирования обратной связи [21]. Однако в массовых онлайн-курсах эти данные редко преобразуются в автоматически создаваемый учебный конспект, который учитывает именно слабые темы конкретного обучающегося.

2 **Фрагментарность чатового формата.** LLM-ассистенты удобны для диалога, но учебный процесс при подготовке к ЕГЭ требует не только ответов на локальные вопросы, но и системного объяснения темы в логически связанной форме. По этой причине чат-ориентированный интерфейс не всегда закрывает потребность в структурированном повторении материала [19, 36].

3 **Риски недостоверности и педагогической несогласованности.** Без внешних ограничений LLM могут генерировать правдоподобный, но методически неудачный или частично некорректный текст. Для образования это критично, поскольку ошибка в объяснении быстро тиражируется и закрепляется у обучающегося [39, 13].

Следовательно, перспективным направлением является объединение трех линий развития: учебной аналитики как источника персонализирующего сигнала, LLM как механизма генерации содержательного текста и агентной оркестрации как способа сделать генерацию многошаговой, контролируемой и воспроизводимой. Именно такая интеграция позволяет перейти от универсального цифрового курса и реактивного чат-бота к сервису, создающему персонализированный учебный конспект как самостоятельный образовательный артефакт.

1.4 Постановка задачи ВКР и методы ее решения

Целью выпускной квалификационной работы является разработка сервиса для веб-ресурса «ЕГЭ Прорыв» [42], который автоматически формирует персонализированные учебные конспекты по предметам ЕГЭ с учетом текущего уровня подготовки ученика, его истории ошибок и динамики освоения тем.

Исходя из проведенного обзора, для достижения поставленной цели необходимо решить следующие задачи:

1 проанализировать существующие подходы к персонализации обучения, генерации учебных материалов и использованию LLM в образовательных сервисах;

2 определить состав пользовательской аналитики, достаточной для адаптации содержания конспекта: изученные темы, типовые ошибки, показатели успеваемости, частоту повторных затруднений и уровень освоения навыков;

3 разработать структуру конспекта, в которой глубина объяснения, выбор примеров, акценты на типичных ошибках и объем повторения зависят от профиля конкретного ученика;

4 спроектировать агентную систему генерации с использованием LangGraph, разделяющую этапы анализа входных данных, планирования структуры конспекта, генерации текста и внутренней проверки результата [23];

5 обеспечить управляемость генерации за счет внешних ограничений предметной области, в том числе опоры на учебные материалы, специфику экзамена и контролируемую подачу контекста модели [24, 13];

6 определить критерии качества получаемого конспекта: релевантность теме, уровень персонализации, логичность структуры, читаемость, фактологическая корректность и пригодность для практического использования в учебном процессе.

Разрабатываемый сервис должен принимать на вход тему или раздел предмета ЕГЭ, профиль обучающегося и агрегированные данные о его взаимодействии с платформой. В качестве входной аналитики могут выступать результаты решения заданий, сведения о частоте ошибок по конкретным типам задач, история прохождения тем, показатели повторного обращения к материалу и иные характеристики, отражающие реальное состояние освоения курса. Выходом системы должен быть связный конспект, который содержит краткое введение в тему, адаптированное объяснение ключевых понятий, разбор типичных ошибок, примеры решения и рекомендации по дальнейшему изучению.

Метод решения поставленной задачи основан на сочетании трех компонентов. Первый компонент — пользовательская аналитика, необходима для построения информативного профиля ученика и выявления содержательных пробелов. Второй компонент — большая языковая модель, обеспечивающая генерацию связного педагогически ориентированного текста [19, 36]. Третий компонент — агентная логика, реализуемая с помощью LangGraph, обеспечивающая разделение ролей внутри конвейера генерации, сохранение состояния и управляемое прохождение этапов анализа, планирования и синтеза ответа [23].

С практической точки зрения разрабатываемая система должна удовлетворять следующим ограничениям:

1 **предметная корректность** — содержание конспекта не должно выходить за рамки школьной программы и формата ЕГЭ;

2 **персонализированность** — объяснение должно учитывать реальные пробелы ученика, а не ограничиваться формальным упоминанием его уровня;

3 **прозрачность логики** — выбор акцентов в конспекте должен быть объясним через доступные аналитические признаки;

4 **безопасность** — система не должна раскрывать чувствительные пользовательские данные и должна минимизировать риск генерации недостоверных рекомендаций;

5 **интегрируемость** — сервис должен быть встроен в уже существующий веб-ресурс без разрушения его текущих рабочих сценариев.

Итак, постановка задачи ВКР может быть сформулирована следующим образом: требуется разработать программный сервис, который на основе пользовательской аналитики и агентно организованной языковой модели генерирует персонализированные учебные конспекты по предметам ЕГЭ для платформы «ЕГЭ Прорыв» [42], обеспечивая адаптацию содержания к индивидуальным дефицитам знаний, соблюдение предметных ограничений и пригодность результата для реального учебного использования.

1.5 Выводы по главе

1 Рынок онлайн-подготовки к ЕГЭ предлагает масштабируемые цифровые курсы и тренажеры, однако в большинстве решений персонализация затрагивает преимущественно выбор заданий и траектории, а не автоматическое создание индивидуализированного теоретического материала.

2 Learning Analytics, Educational Data Mining и Knowledge Tracing дают теоретическую и инструментальную основу для выявления учебных дефицитов, но сами по себе не решают задачу генерации связного учебного конспекта.

3 Большие языковые модели открывают возможность автоматизированного создания адаптивных образовательных текстов, однако требуют внешних ограничений, аналитического контекста и контролируемой оркестрации из-за рисков недостоверности, непрозрачности и педагогической несогласованности.

4 Перспективным решением является сервис, интегрированный в «ЕГЭ Прорыв» [42], который объединяет пользовательскую аналитику, LLM и агентную архитектуру на базе LangGraph для генерации персонализированных конспектов по предметам ЕГЭ.

2 Глава 2. Проектирование и реализация сервиса генерации персонализированных учебных конспектов

2.1 Архитектура сервиса персонализированных конспектов

Разрабатываемый сервис встроен в цифровую образовательную платформу «ЕГЭ Прорыв» [42], на которой пользователь уже взаимодействует с учебными материалами, решает задания, получает обратную связь и формирует цифровой след своей подготовки. Ядро платформы составляет система адаптивного обучения, использующая граф микромодулей, байесовскую оценку компетенций и кривую забывания для построения индивидуальной образовательной траектории [43, 40]. Поскольку в платформе присутствуют сценарии автоматизированной практики, проверки решений и генерации задач, в том числе задач с графиками, новый модуль персонализированных конспектов проектировался не как изолированный прототип, а как отдельный прикладной сервис, использующий общую пользовательскую и предметную инфраструктуру платформы.

Архитектурно решение построено по принципу разделения интерфейсного, прикладного, аналитического и интеллектуального контуров. Такой подход позволяет не смешивать работу пользовательского интерфейса, бизнес-логику платформы, хранение учебных данных и ресурсоемкую генерацию текста. В результате система сохраняет расширяемость и может быть встроена в существующий веб-ресурс без разрушения уже работающих пользовательских сценариев.

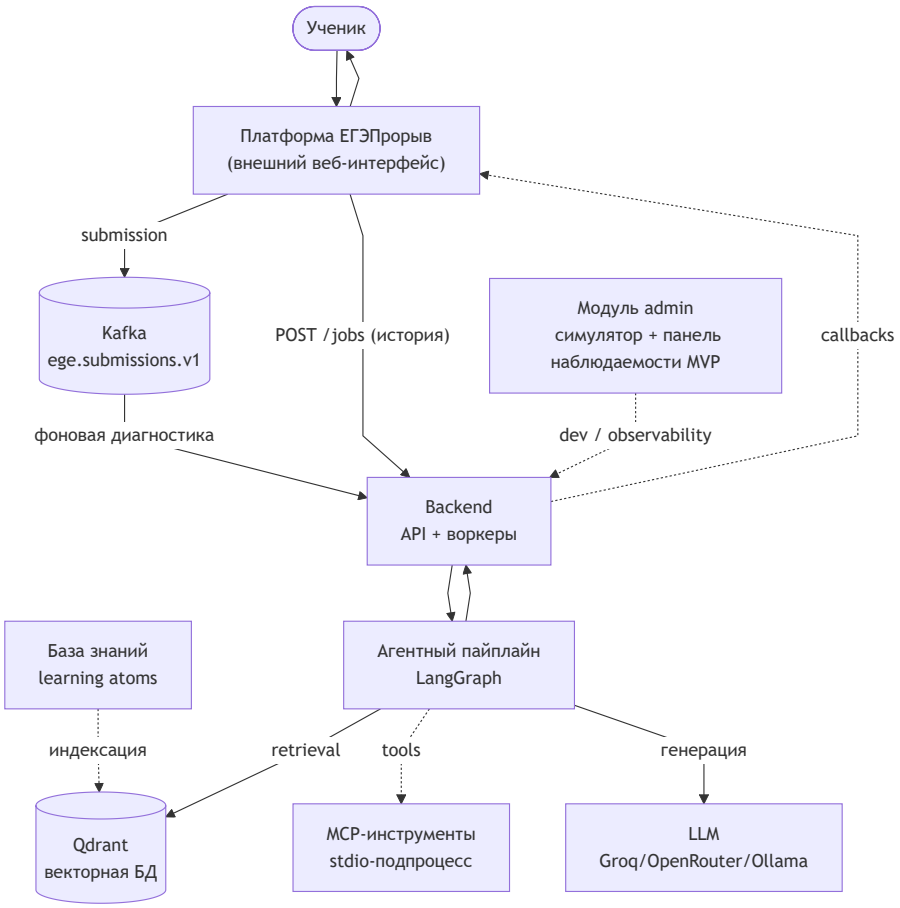

Рисунок 2.1 – Архитектура сервиса персонализированных конспектов (верхнеуровневое представление). Блок *Backend* объединяет HTTP API и фоновых воркеров; их разделение и развёртывание подробно рассмотрены в Главе 3 и Главе 4 (Таблица 4.1). Платформа ЕГЭПрорыв взаимодействует с сервисом по двум каналам: фоновая диагностика ответов через Kafka и явный запрос генерации конспекта через `POST /jobs`. Async-семантика возврата результата (callback / опрос) описана в подразделе 4.4.

Схема на рисунке 2.1 отражает верхнеуровневое взаимодействие компонентов. Основной пользовательский сценарий начинается не во внутреннем интерфейсе разрабатываемого сервиса, а во внешнем веб-интерфейсе платформы «ЕГЭ Прорыв» [42]. Ученик выбирает соответствующий сценарий в платформе, после чего внешний сервис отправляет запрос на генерацию в backend-модуль персонализированных конспектов. Собственный веб-интерфейс данного проекта рассматривается только как вспомогательный контур для разработки, ручной проверки и отладки.

Центральным компонентом выступает программный интерфейс (Application Programming Interface, API) на базе FastAPI [15]. Он принимает запрос от внешнего веб-сервиса, определяет ученика по переданному идентификатору и запускает агентный пайплайн генерации. При этом backend не формирует весь персонализирующий контекст заранее: он передает в пайплайн идентификатор ученика и исходные параметры запроса, а

дальнейший доступ к профилю, учебным материалам и другим данным выполняется через инструменты во время работы агента.

Интеллектуальная часть системы представлена **агентным пайплайном** на базе LangGraph [23]. Внутри пайплайна LLM управляет ходом генерации и при необходимости вызывает доступные инструменты: получение сведений из профиля ученика, поиск релевантных учебных атомов, обращение к retrieval-механизму и извлечение другого контекста. База знаний содержит учебные атомы по заданиям ЕГЭ, которые индексируются в Qdrant и становятся доступны агенту через инструмент поиска по векторному хранилищу. Инструменты предоставляются отдельным MCP-сервисом.

LLM-модуль может использовать Groq, OpenRouter или локальный Ollama. Языковая модель не только формирует текст конспекта, но и по ходу работы решает, какие инструменты нужны для получения дополнительного контекста. После завершения агентного пайплайна backend возвращает готовый результат во внешний веб-интерфейс платформы, где его видит ученик. Дополнительно сформированный конспект и связанные результаты сохраняются в eval-архив, что позволяет в дальнейшем анализировать качество генерации и сравнивать разные версии системы.

Таким образом, архитектура сервиса сочетает три принципиально разные группы данных: предметный контекст, пользовательскую аналитику и генеративную модель. Именно их совместное использование позволяет перейти от универсального справочного текста к индивидуализированному конспекту, ориентированному на дефициты знаний конкретного обучающегося.

2.2 Обоснование выбора методов и инструментальных средств

Выбор методов решения определялся спецификой задачи. С одной стороны, системе требовалось генерировать естественный, логически связный и педагогически понятный текст. С другой стороны, содержание такого текста должно было оставаться привязанным к школьной программе, формату ЕГЭ и истории ошибок конкретного пользователя. По этой причине в работе был выбран комбинированный подход, объединяющий учебную аналитику, RAG, векторный поиск и агентную оркестрацию реализованную на LangGraph.

Использование *Learning Analytics* и элементов Learner Modeling позволяет извлекать из цифрового следа ученика признаки, значимые для персонализации: устойчивые ошибки, неусвоенные темы, характер затруднений и динамику прогресса [21, 1]. Без этого этапа генерация оставалась бы лишь тематической, но не персонализированной. Следовательно,

аналитический контур является не вспомогательной, а содержательно определяющей частью решения.

Выбор RAG объясняется необходимостью снизить вероятность галлюцинаций и сделать итоговый конспект опирающимся на проверенные учебные материалы [24, 16, 13]. В отличие от прямого обращения к LLM без внешнего контекста, RAG-подход позволяет перед генерацией извлекать фрагменты теории, примеры и формулировки, релевантные теме и уровню обучающегося. Для образовательного сервиса это критично, поскольку даже стилистически хороший, но предметно неточный текст не обладает практической ценностью.

В качестве векторной базы данных был выбран Qdrant. Выбор векторной базы данных обусловлен тем, что сервису требуется выполнять не обычный поиск по ключевым словам, а извлечение семантически близких фрагментов учебных материалов по запросу. Qdrant предоставляет модель хранения коллекций векторов с метаданными, поддерживает фильтрацию по метаданным и подходит для задач RAG, где важны как релевантность, так и задержка ответа [31]. По сравнению с хранением материалов только в реляционной форме это дает более устойчивый поиск близких по смыслу фрагментов, даже если формулировки заданий в которых ошибся ученик и учебный материал различаются лексически.

Языковая модель в данной архитектуре используется не как автономный источник знания, а как механизм генеративной сборки итогового текста. Ее задача заключается в том, чтобы на основе уже найденного контекста и профиля ученика сформировать конспект с нужной глубиной объяснения, последовательностью разделов и акцентом на трудных местах. Такой подход соответствует выводам современных исследований об эффективном использовании LLM в образовании: наибольшую практическую пользу дают не “свободные” ответы модели, а управляемые сценарии с внешними ограничениями и прозрачной логикой формирования контекста [19, 39, 36].

Выбор LangGraph в качестве целевого фреймворка оркестрации обусловлен необходимостью формализовать многошаговую агентную логику генерации: отдельные стадии — анализ профиля ученика, извлечение контекста, построение плана конспекта, генерация текста и контроль структуры — должны быть представлены не как набор слабо связанных вызовов, а как единый граф, хранящий состояние между этапами с явным переходом между состояниями [23]. Это упрощает сопровождение сервиса, повышает воспроизводимость результата и позволяет точнее контролировать, на каком именно этапе формируется персонализация.

С инженерной точки зрения выбранный набор средств также оправдан. Backend-оркестрация позволяет отделить пользовательский интерфейс от генеративной логики, а модульная организация сервиса облегчает дальнейшее масштабирование, замену отдельных

компонентов и контроль качества результата. Иными словами, предложенный стек является не только функционально достаточным, но и технологически пригодным для интеграции в действующую образовательную платформу.

2.3 Структура классов и интерфейсов проекта

Внутренняя структура проекта организована как набор относительно независимых слоев: доменного, инфраструктурного, сервисного и интерфейсного. Такое разделение позволяет изолировать предметные сущности от технических деталей хранения, обращения к LLM и HTTP-взаимодействия. В результате один и тот же профиль ученика может использоваться как при обработке API-запроса, так и внутри агентного пайплайна, а замена векторного хранилища или LLM-провайдера не требует изменения доменной логики.

Доменный слой задает основные понятия системы. Класс `StudentProfile` описывает изменяемое состояние обучающегося: историю попыток, число ошибок по подтипам заданий, журнал событий ошибок, оценки межзадачных навыков и служебные временные метки. Его публичный интерфейс включает метод `record_attempt`, который фиксирует новую попытку решения и обновляет производные показатели, методы `error_score` и `mastery` для расчета силы ошибки и уровня освоения подтипа, а также методы `to_dict` и `from_dict` для сериализации профиля. Благодаря этому профиль не является простой JSON-структурой: он инкапсулирует правила обновления учебного состояния, но остается удобным для сохранения в файловом хранилище.

Предметные учебные материалы представлены объектами `Atom`. Каждый учебный атом содержит идентификатор, название, текст, номер задания, связанные подтипы, теги ошибок, пререквизиты и общие навыки. На уровне модуля базы знаний дополнительно поддерживаются коллекции `ATOMS` и `ATOM_BY_ID`, а также функции для обхода графа пререквизитов. Поэтому учебный атом используется одновременно как единица индексирования в Qdrant, как узел графа знаний и как содержательный фрагмент, который может быть включен в контекст генерации.

Слой хранения и извлечения связывает доменную модель с инфраструктурой поиска. Класс `ProfileStore` отвечает за загрузку и сохранение объектов `StudentProfile`, а `QdrantStore` инкапсулирует работу с векторной коллекцией: создание коллекции, пакетную индексацию документов и гибридный поиск. Результат выдачи нормализуется структурой `Retrieved`, содержащей найденный `Atom` и итоговый скор. Центральным компонентом этого слоя является `RagEngine`. Его интерфейс скрывает последовательность технических шагов:

построение эмбеддингов, гибридный поиск, объединение нескольких запросов, персонализированное ранжирование по ошибкам и навыкам ученика, расширение по графу пререквизитов и финальное упорядочивание материалов. За счет этого сервисный слой работает не с деталями Qdrant, а с уже отобранными учебными атомами.

Интерфейс взаимодействия с языковой моделью также вынесен в отдельный контракт. Протокол `ChatClient` задает минимальный метод `chat`, возвращающий нормализованный объект `LlmResponse`. Если модель запрашивает инструменты, каждый такой запрос представляется структурой `ToolCall` с идентификатором, именем инструмента и словарем аргументов. Конкретные классы `GroqClient`, `OpenRouterClient` и `OllamaClient` реализуют этот контракт для разных LLM-бэкендов. Такое решение уменьшает связанность агентного пайплайна с конкретным поставщиком модели: оркестратор работает с общим интерфейсом, а детали HTTP-запросов, формата ответа и потоковой выдачи остаются внутри адаптеров.

Отдельный набор классов описывает инструментальный контур генерации конспекта. `ConspectToolContext` хранит идентификатор ученика, ссылку на `ProfileStore` и экземпляр `RagEngine`; через метод `load_profile` инструменты получают актуальный профиль без прямого доступа к API-слою. Класс `ConspectTool` задает описание одной функции, доступной агенту: имя, текстовое описание, JSON-схему параметров и обработчик вызова. Метод `schema` преобразует это описание в формат function calling. Поверх этих дескрипторов работает `ConspectToolRegistry`, который предоставляет схемы нужных инструментов и диспетчеризует вызовы по имени. В результате LLM получает не произвольный доступ к коду, а ограниченный набор проверяемых операций: получить сводку профиля, найти учебные атомы, определить ближайшие темы, извлечь недавние ошибочные попытки или похожие решенные задания.

Агентная часть проекта использует интерфейс состояния, заданный через `TypedDict`. `SectionAgentState` описывает состояние агента одного раздела конспекта: ключ раздела, пользовательский и системный промпты, клиент LLM, контекст инструментов, историю сообщений, текущий номер шага, ожидающие вызовы инструментов и черновой результат. `ConspectGraphState` задает состояние верхнего графа LangGraph: идентификатор ученика, общий контекст, системный промпт, LLM-клиент, контекст инструментов, словарь черновиков разделов и итоговый текст конспекта. Для поля `draft_sections` используется редьюсер, объединяющий результаты параллельных секционных агентов. Тем самым структура состояния явно фиксирует, какие данные передаются между узлами графа и какие результаты должны быть собраны в финальном узле.

Внешний HTTP-контракт сервиса задается Pydantic-схемами. `ConspectResponse` опи-

сывает результат генерации и содержит текст конспекта, названия найденных источников и идентификаторы ближайших учебных атомов. `ConspectContextResponse` используется для диагностического сценария, когда требуется получить не итоговый текст, а собранный контекст персонализации: область задачи, уровень детализации, блок ошибок и итоговую секцию контекста. Эти схемы отделяют внутреннюю структуру пайплайна от формата ответа API и делают контракт интеграции с внешней платформой явным.

Таким образом, структура классов проекта отражает общую архитектурную идею сервиса. `StudentProfile` и `Atom` задают предметную основу персонализации, `ProfileStore`, `QdrantStore` и `RagEngine` обеспечивают хранение и извлечение релевантного контекста, `ChatClient` и его реализации отделяют генеративную модель от бизнес-логики, а `ConspectToolRegistry` и состояния LangGraph формируют управляемый агентный контур. Такая декомпозиция делает систему расширяемой: можно добавлять новые инструменты, подключать другой LLM-бэкенд или изменять стратегию извлечения без переписывания API-контракта и доменной модели.

2.4 Алгоритм формирования персонализированного конспекта

Алгоритм формирования конспекта в текущей реализации начинается с запроса генерации конспекта. Она принимает идентификатор ученика (по умолчанию используется демонстрационный профиль) и через механизм зависимостей FastAPI [15] получает два заранее инициализированных объекта: хранилище профилей и RAG-движок. Эти объекты создаются один раз при запуске приложения и сохраняются в состоянии FastAPI-приложения [15], поэтому при каждом запросе не требуется заново индексировать материалы или загружать инфраструктурные компоненты.

Дальнейшая обработка передается из API-слоя в сервисную функцию генерации конспекта. Важно, что ручка не формирует текст самостоятельно: она только инициирует пайплайн, обрабатывает ошибки LLM-бэкенда и после успешной генерации возвращает ответ в виде структуры с тремя полями: текст конспекта, названия найденных источников и идентификаторы ближайших учебных атомов.

1 **Получение запроса и внедрение зависимостей.** Пользователь обращается к маршруту `/conspect`, передавая идентификатор ученика. FastAPI [15] подставляет в обработчик экземпляры хранилища пользователей и RAG-движка. На этом уровне не выполняется ни извлечение контекста, ни обращение к LLM: API-слой только подготавливает входные зависимости и вызывает сервис генерации.

2 **Запуск сервисного слоя.** Обработчик вызывает функцию, внутри которой создается замыкание для сборки контекста ученика, формируется объект контекста с идентификатором ученика, хранилищем профилей и RAG-движком, а также создается клиент языковой модели. Выбирается активный LLM-бэкенд определенный в настройках: поддерживаются Groq, OpenRouter и локальный Ollama. Если для выбранного облачного бэкенда не задан API-ключ, сервисная функция порождает ошибку доступности, которая на уровне ручки превращается в HyperText Transfer Protocol (HTTP) код 503.

3 **Подготовка начального состояния LangGraph.** Оркестратор создает начальное состояние графа: идентификатор ученика, системный промпт из YAML-конфигурации, клиент LLM, контекст инструментов и пустой словарь будущих разделов. Если клиент модели поддерживает вызов инструментов, в состояние помещается не заранее собранный полный извлеченный контекст, а краткий каркас: модель должна сама запросить профиль, ошибки, ближайшие темы и учебные атомы через доступные инструменты. Если инструментальный режим недоступен, сервис заранее строит контекст: загружает профиль ученика, выделяет последние ошибочные попытки, формирует по ним поисковые запросы, выполняет извлечение контекста и собирает блоки промпта.

4 **Извлечение персональных данных и учебного контекста.** В инструментальном режиме каждый агент раздела может обращаться к инструментам: получить краткую сводку профиля, последние ошибочные попытки, типичные заблуждения, ближайшие темы для изучения, похожие решенные задачи и релевантные учебные атомы. Также в модели доступен инструмент который позволяет использовать текущий профиль ученика и RAG-движок. Поисковая часть выполняет гибридный поиск в векторной базе данных, объединяет результаты разных запросов, усиливает атомы, связанные с ошибками и слабыми навыками ученика, штрафует уже хорошо освоенные темы, расширяет выборку по графу пререквизитов и затем отбирает разнообразный набор материалов.

5 **Разделение задачи на шесть специализированных разделов.** Граф не просит модель сразу написать весь конспект одним ответом. На старте он выполняет декомпозицию и отправляет шесть параллельных задач, каждая из которых отвечает за генерацию одного раздела конспекта: “Что важно запомнить”, “Типичные ошибки”, “Алгоритм решения”, “Разбор примера”, “Найди ошибку” и “Чеклист перед ответом”. Каждый раздел конспекта генерируется специализированным ReAct (Reasoning Action) агентом рисунок 2.2. Для каждого раздела используется отдельный пользовательский промпт из

конфигурационного YAML-файла, но все разделы получают общий системный промпт, один и тот же идентификатор ученика и доступ к инструментам.

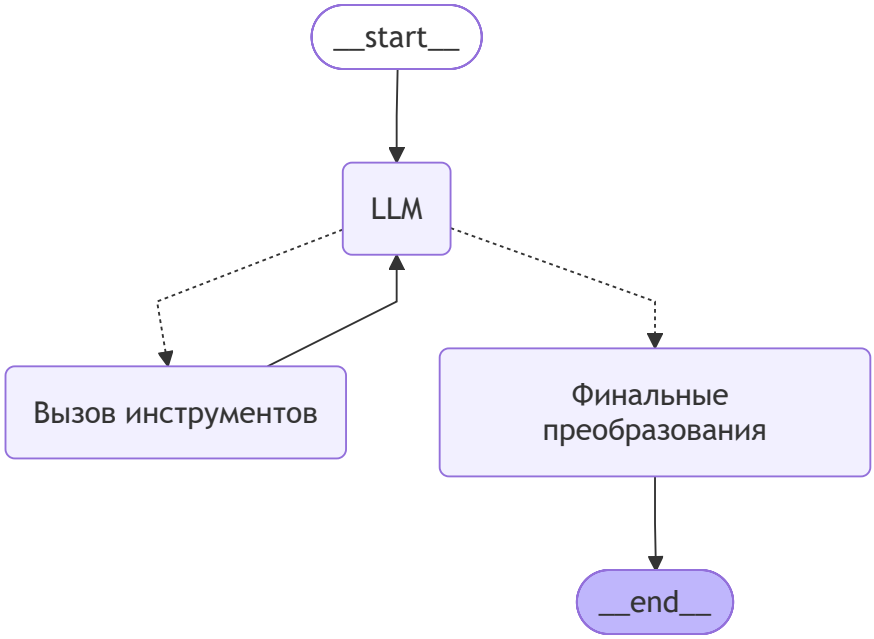

Рисунок 2.2 – Схема Агента отвечающего за генерацию одного раздела конспекта.

6 **ReAct-цикл внутри агента раздела.** Каждый раздел генерируется подграфом, который представлен на рисунке 2.2. Сначала модель получает системный и пользовательский промпты. Если доступны инструменты, к запросу добавляется инструкция не выдумывать профиль, ошибки и справочный материал, а запросить их явно. Модель может вернуть один или несколько вызовов инструментов; система выполняет эти вызовы, добавляет результаты в историю сообщений и снова обращается к модели. Такой цикл ограничен несколькими раундами, после чего модель обязана сформировать итоговый текст раздела без технических деталей и внутренних идентификаторов.

7 **Формирование черновиков разделов.** После завершения генерации раздела его текст очищается от служебных фрагментов, например от блоков внутреннего рассуждения или технических идентификаторов учебных атомов. Затем к разделу добавляется фиксированный заголовок, а результат записывается в общее состояние графа. Так как разделы генерируются параллельно, для этого поля используется редьюсер, который объединяет результаты нескольких веток без конфликта обновлений состояния.

8 **Сборка итогового конспекта.** После завершения всех параллельных веток граф переходит в узел сборки конспекта. Этот узел не вызывает LLM, а детерминированно

объединяет готовые разделы в фиксированном порядке: ключевые факты, типичные ошибки, алгоритм решения, пример, задание на поиск ошибки и чеклист. Результат сохраняется в поле итогового конспекта.

9 **Возврат результата из графа в сервис.** Сервис получает из финального состояния графа текст итогового конспекта, а также списки извлеченных атомов и ближайших тем для изучения. Эти списки предусмотрены контрактом ответа и используются при сохранении результата для последующей оценки, однако в текущем инструментальном графе основным возвращаемым артефактом является именно собранный текст конспекта.

10 **Сохранение результата для оценки.** После успешной генерации API-ручка пытается сохранить конспект в валидационный архив. Сохраняется markdown-файл с текстом конспекта и JSON-файл с метаданными: идентификатор ученика, время генерации, основные ошибки, ближайшие темы для изучения и последние ошибочные попытки. Ошибка на этом этапе не ломает пользовательский сценарий: она логируется как предупреждение, а сгенерированный конспект все равно возвращается пользователю.

11 **Формирование HTTP-ответа.** На последнем шаге обработчик создает объект ответа. И возвращает его пользователю.

Таким образом, отличие текущей реализации от простого вызова LLM состоит не только в наличии RAG-контекста, но и в способе оркестрации. Сервис декомпозирует конспект на устойчивые смысловые разделы, генерирует их параллельно, позволяет каждому разделу при необходимости обращаться к данным профиля и базе знаний, а затем собирает результат детерминированным узлом графа. Персонализация возникает из двух источников: из аналитического профиля ученика, который показывает, какие ошибки и темы требуют внимания, и из контура извлечения учебного контекста, который ограничивает генерацию релевантным учебным материалом. Благодаря этому модель выбирает не только *что* объяснять конкретному ученику, но и *как* это объяснение должно быть представлено: подробно, кратко, через пример, через предупреждение о типичной ошибке или через чеклист самопроверки.

2.5 Реализация сервиса и решения, обеспечивающие масштабируемость

Так как ВКР относится к программным проектам, ориентированным на создание работоспособного и потенциально высоконагруженного приложения, при проектировании сервиса

необходимо учитывать не только качество генерации, но и эксплуатационные характеристики решения. Для образовательной платформы это особенно важно, поскольку обращения к сервису могут происходить массово: перед экзаменом возрастает интенсивность повторения тем, пользователи параллельно решают задания, просматривают аналитику и запрашивают новые материалы.

Базовая производительность сервиса достигается уже за счет самой логики RAG. Учебные материалы индексируются заранее, а в момент пользовательского запроса система выполняет векторный поиск только по готовому векторному представлению. Это существенно дешевле и быстрее, чем каждый раз передавать в модель большой объем “сырого” текста. Следовательно, вынесение базы знаний в Qdrant одновременно повышает релевантность контекста и снижает вычислительную нагрузку на генеративный слой.

Для снижения времени отклика и уменьшения числа повторных обращений к тяжелым компонентам архитектура сервиса включает кэширование часто используемых сущностей: шаблонов промптов, фрагментов учебного контекста, промежуточных аналитических представлений и результатов повторных запросов по близким темам. Такое решение особенно эффективно в образовательном домене, где большое число пользователей обращается к сходным разделам программы и типовым ошибкам.

Следующим важным решением является разделение интерактивного и фонового контуров обработки. Запрос пользователя должен быстро пройти валидацию, получить идентификатор задачи и быть передан в обработку без блокировки интерфейса. Ресурсоемкие операции — генерация длинного текста, формирование эмбеддингов для новых материалов, обновление индексов, агрегация журналов — рационально выполнять асинхронно. Это позволяет не перегружать основной web-контур и поддерживать стабильную работу платформы даже при росте числа одновременных пользователей.

Для масштабирования по числу запросов прикладной слой сервиса целесообразно сохранять stateless. В этом случае горизонтальное масштабирование backend-экземпляров, балансировка входящего трафика и вынос состояния в специализированные хранилища становятся естественным продолжением архитектуры. Аналогично векторный поиск на базе Qdrant может масштабироваться отдельно от web-слоя и LLM-модуля, что важно при неравномерной нагрузке, когда число поисковых обращений существенно превышает число генеративных вызовов.

Не менее значимым является контур наблюдаемости. Для промышленной эксплуатации необходимо собирать метрики времени ответа, частоты ошибок, загрузки очередей, числа обращений к модели, качества извлеченного контекста и доли неуспешных генераций. Эти

данные позволяют не только поддерживать стабильность сервиса, но и совершенствовать персонализацию на основе реальных пользовательских сценариев. Таким образом, масштабируемость в данной работе понимается не как единичное аппаратное усиление, а как архитектурное свойство системы, обеспечиваемое раздельным масштабированием компонентов, кэшированием, асинхронной обработкой и постоянным мониторингом.

2.6 Результат решения задачи и личный вклад участников проекта

Результатом выполненной работы является архитектурно и методически оформленный сервис генерации персонализированных учебных конспектов, интегрируемый в платформу «ЕГЭ Прорыв» [42]. Данный сервис использует учебную аналитику пользователя, извлечения контекста по базе знаний и большую языковую модель для автоматического формирования связного учебного текста, адаптированного к текущему уровню подготовки обучающегося.

Настоящая выпускная квалификационная работа является совместной; распределение задач между участниками проекта приведено ниже.

Ахундов А.

1 Прототип RAG-системы генерации персонализированных конспектов (Глава 3): модель учебных атомов с графом пререквизитов и тегами ошибок, хранение пользовательских профилей в формате JSON, первичная реализация локального векторного поиска, многоуровневое ранжирование с профильным бустом и степенным затуханием баллов ошибок, MMR-диверсификация и конвейер генерации конспекта с многоступенчатой верификацией.

2 Внедрение разработанного сервиса в действующую веб-платформу ЕГЭПрорыв (Глава 4): контракты взаимодействия (OpenAPI-спецификации и схема Kafka-события), реализация двух асинхронных потоков обработки (фоновая диагностика ответов через Kafka и генерация конспекта по HTTP-запросу), семантика идемпотентности и классификация ошибок на постоянные и временные с экспоненциальным backoff'ом, машинная аутентификация JWT-HS256 и развёртывание в Kubernetes с локальным симулятором платформы для отладки.

Жарковский Д.

1 Тюнинг RAG-системы и переход на Qdrant (Глава 3): миграция векторного поиска с прототипной локальной реализации на полноценную векторную базу данных Qdrant,

настройка коллекции и payload-индексов, конфигурация гибридного поиска через Prefetch+RRF, подбор параметров ранжирования (веса каналов, константа RRF, бонусы графа пререквизитов, порог mastery, параметр затухания) для максимизации качества выдачи.

2 Интеграция агентной оркестрации на основе LangChain и LangGraph (Глава 3): построение пайплайна генерации конспекта как stateful-графа состояний, реализация tool-вызовов через MCP-инструменты, согласование подсистем retrieval, планирования структуры конспекта и верификации в едином управляемом конвейере.

3 Метрики качества и оценочный конвейер: реализация бенчмарков retrieval (precision/recall/NI для заданий 6, 10 и 12) и метрики LLM-judge для оценки доли релевантных атомов в контексте конспекта; полученные значения приведены в Главе 3.

Практическая ценность разработанного решения состоит в том, что оно позволяет перевести персонализацию теоретического материала из ручного режима в автоматизированный. Платформа получает возможность не только фиксировать ошибки пользователя и подбирать ему задания, но и формировать индивидуальный теоретический материал, непосредственно направленный на устранение обнаруженных дефицитов. Тем самым повышается полезность накопленной учебной аналитики и расширяется набор образовательных сценариев, доступных пользователю в одном цифровом контуре.

Научно-прикладная новизна решения заключается в интеграции нескольких подходов в едином сервисе: аналитики учебных данных, RAG и векторного поиска по предметной базе знаний. В отличие от типичного чатового сценария взаимодействия с LLM, здесь результатом является не одиночный ответ на вопрос, а полноформатный персонализированный конспект, создаваемый в воспроизводимом и управляемом конвейере.

2.7 Выводы по главе

1 Во второй главе предложена архитектура сервиса персонализированных учебных конспектов, интегрируемого в платформу «ЕГЭ Прорыв» [42] и использующего пользовательскую аналитику, RAG и LLM в рамках единого прикладного контура.

2 Обосновано, что применение RAG-подхода и векторной базы данных Qdrant позволяет связать генерацию конспекта с проверенными учебными материалами, повысить релевантность контекста и снизить риск предметных неточностей.

3 Показано, что персонализация достигается не только за счет генеративной модели, но прежде всего за счет анализа цифрового следа обучающегося, выделения устойчивых ошибок и последующей адаптации структуры объяснения к профилю ученика.

4 Предложенный алгоритм формирования конспекта обладает элементами научно-прикладной новизны, поскольку объединяет в одном воспроизводимом пайплайне Learner Modeling, извлечение из векторной базы знаний и управляемую генерацию структурированного образовательного текста.

5 Практическая ценность разработанного решения состоит в возможности промышленной интеграции сервиса в высоконагруженную образовательную платформу, где наряду с качеством генерации учитываются масштабируемость, наблюдаемость, асинхронная обработка и устойчивость к росту пользовательской нагрузки.

3 Глава 3. Реализация подсистемы извлечения и генерации персонализированных конспектов

В предыдущей главе были описаны архитектура сервиса и обоснование выбора основных методов. Данная глава раскрывает техническую реализацию ключевой подсистемы — RAG-пайплайна, отвечающего за извлечение релевантного учебного контекста и генерацию персонализированного конспекта. Последовательно рассматриваются: структура предметной базы знаний, модели векторного представления и механизм индексирования, алгоритм гибридного поиска с многоуровневым ранжированием, а также конвейер генерации и верификации итогового текста.

3.1 Структура предметной базы знаний

Предметная база знаний является центральным источником доверенного контента для RAG-пайплайна. В отличие от классического подхода к RAG, при котором корпус формируется путем автоматического разбиения (chunking) длинных документов на фрагменты фиксированного размера, в данной работе используется экспертно структурированная база знаний, состоящая из *атомов* — минимальных тематически целостных единиц учебного контента. Концепция атомов развивает идею *микромодулей*, предложенную в [43] и формализованную в [40]: каждый микромодуль представляет собой элементарную образовательную единицу, связанную с набором компетенций, а совокупность микромодулей образует ориентированный граф зависимостей. В нашей работе атомы сохраняют аналогичную графовую структуру пререквизитов, но дополнительно размечаются тегами ошибок и межзадачными навыками, что делает их пригодными не только для адаптации траектории обучения, но и для адресного извлечения контекста при генерации конспекта.

Каждый атом представляет собой запись со следующими полями:

1 **id** — уникальный идентификатор, кодирующий номер задания и порядковый номер (например, `t06_mm01`, `t10_mm51`, `t12_mm03`);

2 **title** — краткое название темы микромодуля;

3 **text** — учебный текст с формулами, определениями, примерами решений и указаниями на типичные ошибки;

4 **task_number** — номер задания ЕГЭ (6, 10 или 12);

5 **subtypes** — список подтипов задач, к которым относится атом (например, `exponential_eq`, `motion`, `polynomial_extrema`);

6 **error_tags** — перечень тегов типичных ошибок, связанных с данным атомом (например, `deriv_forgot_chain_rule`, `river_swap_plus_minus`);

7 **prerequisites** — список идентификаторов атомов-пререквизитов, образующих направленный ациклический граф зависимостей;

8 **shared_skills** — метки межзадачных навыков (например, `equation_setup`, `sign_analysis`), позволяющих связывать атомы из разных заданий ЕГЭ по общей компетенции.

Атомы хранятся в формате YAML в отдельных файлах для каждого задания: `atoms_t6.yaml`, `atoms_t10.yaml`, `atoms_t12.yaml`. Дополнительно предусмотрен файл `misconceptions.yaml`, содержащий экспертно описанные типичные заблуждения учеников. Каждая запись о заблуждении привязана к конкретному тегу ошибки и включает текстовое описание некорректного рассуждения, его причины и мнемоническое правило для запоминания верного подхода. Заблуждения загружаются в общий массив атомов и индексируются наравне с основным учебным контентом, что позволяет системе при обнаружении у ученика конкретной ошибки извлекать не только теоретический материал, но и адресное разъяснение заблуждения.

Совокупность атомов формирует направленный ациклический граф (DAG) пререквизитов. Связь «атом A является пререквизитом атома B» означает, что для понимания материала B необходимо предварительное усвоение материала A. Такая структура используется в процессе retrieval для двух целей. Во-первых, если для ученика релевантен атом B с высоким ранжирующим баллом, система автоматически добавляет пререквизитные атомы A в набор извлеченных фрагментов, обеспечивая логическую полноту объяснения. Во-вторых, при формировании итогового конспекта атомы упорядочиваются топологической сортировкой по графу пререквизитов, так что пререквизитные понятия предшествуют зависимым.

Помимо прямых пререквизитных связей, механизм *shared skills* обеспечивает межзадачный перенос (cross-task transfer). Ряд навыков — составление уравнения, проверка ограничений, анализ знака — используются одновременно в заданиях №6, №10 и №12, хотя предметный контекст различается. Четыре группы shared skills (`equation_setup`, `constraint_checking`, `sign_analysis`, `answer_plausibility`) объединяют атомы из разных заданий, позволяя профильному бусту учитывать успехи и ошибки ученика по смежным компетенциям.

Выбор атомарной структуры базы знаний вместо автоматического chunking обусловлен спецификой образовательного домена. Во-первых, каждый атом содержит тематически замкнутое объяснение, что устраняет проблему «разрезания» связного определения или примера на границе чанка. Во-вторых, экспертная разметка тегами ошибок и подтипами задач создает структуру метаданных, недоступную при автоматическом разбиении, и позволяет строить более точные фильтры на этапе поиска. В-третьих, граф пререквизитов обеспечивает логическую последовательность контекста, которая невозможна в плоском корпусе чанков.

3.2 Векторное представление и индексирование учебных материалов

Для обеспечения семантического поиска по базе знаний каждый атом представляется в виде двух типов векторных эмбеддингов: плотного (dense) и разреженного (sparse). Совместное использование двух представлений лежит в основе гибридного поиска, описанного в следующем разделе.

Плотные эмбеддинги. Для получения плотных векторных представлений используется модель `mxbai-embed-large-v1`, разработанная mixedbread.ai [26], с размерностью вектора 1024. Загрузка и инференс модели осуществляются через библиотеку fastembed [30], которая обеспечивает эффективную пакетную обработку без внешних API-зависимостей. После получения эмбеддингов выполняется L2-нормализация:

$$\hat{\mathbf{v}}_i = \frac{\mathbf{v}_i}{\max\bigl(\|\mathbf{v}_i\|_2,\ 10^{-12}\bigr)}, \tag{1}$$

где $\mathbf{v}_i$ — исходный вектор i-го документа. Нормализация гарантирует, что косинусное сходство между запросом и документом вычисляется как скалярное произведение нормализованных векторов.

Разреженные эмбеддинги. Для лексического компонента поиска используется модель `Qdrant/bm25` из библиотеки fastembed [30], реализующая BM25-подобное взвешивание термов [34]. Разреженное представление содержит индексы ненулевых термов и их весовые коэффициенты, что позволяет эффективно выполнять точное терм-совпадение по лексическому содержимому документов. Такое представление дополняет семантический поиск, обеспечивая высокую точность при совпадении ключевых терминов (например, «дискриминант», «логарифм»), которые могут быть размыты при исключительно семантическом сопоставлении.

Хранение в Qdrant. В качестве векторной базы данных используется Qdrant [31]. Коллекция создается с двумя именованными векторными пространствами:

1 `dense` — плотные векторы размерности 1024 с метрикой косинусного расстояния;

2 `sparse` — разреженные BM25-векторы переменной размерности.

Для каждой точки в коллекции сохраняется payload — набор метаданных, включающий идентификатор атома (`doc_id`), полный текст документа, номер задания (`task_number`), список подтипов (`subtypes`), теги ошибок (`error_tags`), заголовок атома и хеш содержимого (`content_hash`). Для полей `task_number` и `subtypes` создаются payload-индексы, что позволяет выполнять серверную фильтрацию при поиске без перебора всех точек коллекции.

Механизм инкрементального обновления. При инициализации RAG-движка вычисляется SHA-256 хеш от текстового содержимого всех атомов. Если в Qdrant уже существует коллекция с совпадающим числом точек, размерностью плотного вектора и хешем контента, повторное эмбеддирование и загрузка пропускаются. В противном случае коллекция пересоздается и данные загружаются пакетами. Такой подход сокращает время холодного старта сервиса при неизменной базе знаний и гарантирует консистентность индекса при обновлении контента.

Формирование текста документа. Перед эмбеддированием для каждого атома формируется текст документа в формате «заголовок + текст»: конкатенация полей `title` и `text` через символ новой строки. Заголовок включается в документ, поскольку содержит ключевые понятия (например, «Показательное уравнение с одинаковым основанием»), важные для семантического сопоставления с запросом.

3.3 Гибридный поиск и многоуровневое ранжирование

Процесс извлечения релевантного контекста реализован как многоэтапный пайплайн (Рисунок 3.1), включающий формирование запросов из профиля ученика, гибридный поиск в Qdrant, межзапросное слияние, профильный буст, расширение по графу пререквизитов, диверсификацию через максимальную предельную релевантность (Maximal Marginal Relevance, MMR) и топологическую сортировку.

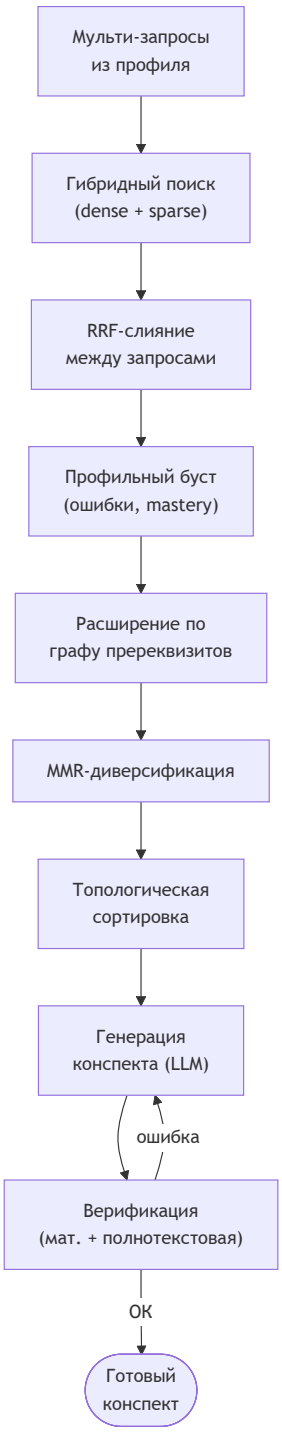

Рисунок 3.1 – Упрощённая схема этапов RAG-пайплайна извлечения и ранжирования контекста.

Формирование поисковых запросов из профиля. В отличие от типичного RAG, где запрос формулируется пользователем, в данной системе множество поисковых запросов генерируется автоматически на основе профиля ученика. Стратегия зависит от наличия данных. Если в профиле присутствуют недавние неверные попытки, каждая из них преобразуется в текстовый запрос вида «ЕГЭ задание N: [условие задачи], ученик ответил X, правильно Y». Дополнительно добавляются запросы из пулов микромодулей (*micromodule*

queries), формируемые из текстов задач, в которых ученик допускал ошибки. При отсутствии данных об ошибках используется *cold start*-стратегия: детерминированная (по хешу идентификатора ученика) выборка из предзаданных пулов запросов для каждого задания. Совокупность запросов дедуплицируется и передается в RAG.

Гибридный поиск в Qdrant. Для каждого текстового запроса вычисляются плотный и разреженный эмбеддинги, после чего выполняется гибридный поиск через механизм Qdrant Prefetch + RRF [31]. Запрос к Qdrant содержит два блока предварительной выборки (prefetch): один по плотному вектору с метрикой косинусного сходства, другой — по разреженному вектору. Каждый prefetch возвращает до $L \cdot p$ кандидатов, где L — целевое число результатов, p — множитель prefetch. Результаты двух потоков объединяются на сервере Qdrant с помощью взвешенного Reciprocal Rank Fusion (RRF) [10]:

$$\mathrm{RRF}(d) = \frac{w_{\mathrm{dense}}}{k + r_{\mathrm{dense}}(d)} + \frac{w_{\mathrm{sparse}}}{k + r_{\mathrm{sparse}}(d)}, \tag{2}$$

где $r_{\mathrm{dense}}(d)$ и $r_{\mathrm{sparse}}(d)$ — ранги документа d в плотном и разреженном списках соответственно, k — константа сглаживания (по умолчанию $k = 60$), w_{dense} и w_{sparse} — весовые коэффициенты каналов (по умолчанию $w_{\mathrm{dense}} = 0{,}40$, $w_{\mathrm{sparse}} = 0{,}25$). При наличии фильтров по номеру задания или подтипу задачи серверная фильтрация применяется до ранжирования, что существенно сокращает множество кандидатов.

Межзапросное RRF-слияние (RAG-Fusion). Поскольку для одного ученика генерируется несколько поисковых запросов, результаты гибридного поиска по всем запросам объединяются вторым уровнем RRF:

$$S_{\mathrm{fused}}(d) = \sum_{j=1}^{Q} \frac{1}{k + r_j(d)}, \tag{3}$$

где Q — число запросов, $r_j(d)$ — ранг документа d в результатах j-го запроса. Документы, встречающиеся в результатах нескольких запросов, получают более высокий итоговый балл, что естественным образом отражает устойчивость релевантности фрагмента для данного ученика.

Профильный буст. После межзапросного слияния итоговый балл каждого документа корректируется на основе профиля ученика:

$$S(d) = S_{\mathrm{fused}}(d) + \gamma \cdot \Big(\mathrm{err}(d) + \beta \cdot \mathrm{skill}(d)\Big) - \mathbb{1}\big[m(d) > \theta_m\big] \cdot \delta_m, \tag{4}$$

где γ — вес профильного компонента ($\gamma = 0{,}30$); $\mathrm{err}(d)$ — суммарный балл ошибок по тегам, связанным с атомом d; $\mathrm{skill}(d)$ — суммарный балл по shared skills атома; β — множитель навыковой компоненты ($\beta = 0{,}3$); $m(d)$ — уровень освоения атома; θ_m — порог mastery ($\theta_m = 0{,}8$); δ_m — штраф за освоенный атом ($\delta_m = 0{,}15$).

Балл ошибки $\mathrm{err}(d)$ вычисляется по журналу событий профиля с применением степенного затухания (power-law decay):

$$\mathrm{error_score}(t) = \sum_{e \in \mathcal{E}_t} w_e \cdot (1 + \Delta_e)^{-\alpha}, \tag{5}$$

где $\mathcal{E}_t$ — множество событий ошибки с тегом t, w_e — вес события, Δ_e — разность между текущим номером попытки и номером попытки события, $\alpha = 0{,}5$ — параметр затухания. Степенной закон обеспечивает постепенное забывание старых ошибок, при этом недавние ошибки вносят доминирующий вклад в балл. Данный механизм перекликается с экспоненциальной кривой забывания, используемой в адаптивном модуле платформы для оценки вероятности владения компетенцией [43]: там $p_k(t) = p_k(t_0) \cdot e^{-\lambda(t-t_0)}$ моделирует спад владения компетенцией при отсутствии практики, а в [40] этот подход дополнен кластеризацией родственных задач и пересчётом вероятностей после каждой попытки. В отличие от этих задач, наш decay применяется не к вероятности владения, а к *весу* ошибки при ранжировании контента: цель состоит не в том, чтобы оценить уровень знаний, а в том, чтобы обеспечить приоритет актуальных проблем в контексте конспекта. Механизм mastery gating подавляет ранжирование атомов, которые ученик, по оценке системы, уже достаточно освоил, направляя контекст конспекта на актуальные пробелы.

Расширение по графу пререквизитов. Для обеспечения логической полноты конспекта система выполняет расширение набора кандидатов по графу атомов. Для каждого из топ-k атомов в текущем рейтинге извлекаются пререквизиты до глубины 2 (обход BFS) и непосредственные зависимые (dependents). Пререквизит получает бонус к рангу, пропорциональный баллу родительского атома, с учетом «слабости» (atom weakness) ученика по данному пререквизиту:

$$S(p) \leftarrow \max\bigl(S(p),\ S(a) + b_{\mathrm{prereq}} \cdot (1 + w_p) + b_{\mathrm{tag}} \cdot |\mathrm{tags}(p)|\bigr), \tag{6}$$

где a — родительский атом, p — пререквизит, b_{prereq} и b_{tag} — конфигурируемые бонусы, w_p — слабость ученика по атому p. Для пререквизитов из другого задания ЕГЭ (cross-task) применяется понижающий множитель. Зависимые атомы (dependents) получают фиксированный

бонус, если удовлетворяют фильтрам по заданию и подтипу.

MMR-диверсификация. После объединения базовых и расширенных кандидатов финальный отбор выполняется алгоритмом Maximal Marginal Relevance [7]:

$$\mathrm{MMR}(d_i) = \lambda \cdot \hat{S}(d_i) - (1 - \lambda) \cdot \max_{d_j \in \mathcal{S}} \mathrm{sim}(d_i, d_j), \tag{7}$$

где $\hat{S}(d_i)$ — нормализованная в $[0, 1]$ релевантность, $\mathcal{S}$ — уже отобранные атомы, $\mathrm{sim}(d_i, d_j)$ — сходство Жаккара по множествам подтипов задач, $\lambda = 0{,}6$. MMR-отбор обеспечивает тематическое разнообразие конспекта: если в топе ранжирования оказалось несколько атомов одного и того же подтипа, алгоритм отдает предпочтение атомам из менее представленных подтипов, сохраняя баланс между релевантностью и полнотой охвата.

Топологическая сортировка. Итоговый набор из k отобранных атомов (по умолчанию $k = 12$) упорядочивается топологической сортировкой по графу пререквизитов. Алгоритм выполняет рекурсивный обход: для каждого атома сначала посещаются его пререквизиты, уже входящие в отобранный набор, затем сам атом. В результате в контексте промпта базовые понятия предшествуют зависимым, что воспроизводит дидактически обоснованную последовательность изложения.

3.4 Генерация и верификация конспекта

На основе профиля ученика и извлеченного контекста RAG-пайплайн переходит к этапу генерации текста конспекта. Этот этап включает сборку структурированного промпта, вызов языковой модели, верификацию математического содержания и постобработку результата.

Сборка промпта. Промпт конспекта состоит из системного сообщения и пользовательского сообщения. Оба формируются на основе шаблонов, хранящихся в YAML-файле `conspect_prompts.yaml`. Пользовательское сообщение включает:

1 указание на задание и подтип;

2 уровень детализации, определяемый автоматически по точности ответов ученика (подробный при точности ниже 30%, средний — от 30% до 60%, краткий — выше 60%);

3 блок ошибок ученика с расшифровкой до 5 наиболее значимых тегов ошибок в человекочитаемом формате, конкретными примерами неверных ответов и условиями задач;

4 извлеченный контекст, разделенный на справочный материал и блок типичных заблуждений;

5 темы-фронтиры — атомы, для которых ученик демонстрирует слабость, но пререквизиты уже освоены, что указывает на зону ближайшего развития;

6 инструкцию по разнообразию примеров, предотвращающую генерацию шаблонных задач.

При включенном режиме структурированного вывода (structured output) к системному промпту добавляется JSON-инструкция, предписывающая модели возвращать конспект в виде объекта с именованными секциями. Это позволяет применять верификацию к каждой секции отдельно.

Генерация. Для генерации конспекта используется языковая модель, доступная через один из поддерживаемых бэкендов: Groq, OpenRouter или Ollama. Выбор бэкенда определяется конфигурацией, что позволяет переключаться между облачными и локальными моделями без изменения логики пайплайна. Генерация выполняется с температурой $T = 0{,}25$ и ограничением длины 8192 токена.

Верификация и ремонт. После генерации черновика конспект проходит многоступенчатую верификацию:

1 **Структурированная верификация (optional).** Если генерация выполнена в JSON-режиме, каждая математическая секция (формулы, примеры, чеклист) проверяется отдельным вызовом LLM с системным промптом, настроенным на поиск арифметических и алгебраических ошибок.

2 **Полнотекстовая верификация.** Второй вызов LLM с низкой температурой ($T = 0{,}1$) проверяет весь конспект на полноту обязательных смысловых блоков, предметную корректность и педагогическую согласованность.

3 **Математическая верификация.** Отдельный проход проверяет вычисления, знаки и формулы на фактическую корректность.

4 **Валидация формата.** Регулярные выражения проверяют конспект на наличие нарушений формата ЕГЭ: ответы в виде десятичных дробей вместо обыкновенных, некорректные обозначения, пропущенные единицы измерения. Обнаруженные нарушения передаются LLM для автоматического ремонта с сохранением структуры и заголовков конспекта.

Постобработка. Финальная постобработка включает удаление из текста конспекта внутренних идентификаторов атомов, технических терминов пайплайна (если они «просочились» из контекста в генерацию), а также блоков «мышления» модели. При обнаружении в финальном ответе JSON-формата выполняется принудительная конвертация в Markdown, поскольку пользователь должен получать конспект исключительно в читаемом текстовом формате.

3.5 Выводы по главе

1 Предметная база знаний реализована как экспертно структурированный набор атомов с графом пререквизитов, тегами ошибок и межзадачными навыками, что обеспечивает тематическую целостность единиц поиска и возможность адресного извлечения контента по профилю ученика.

2 Двухканальное векторное представление (плотные эмбеддинги `mxbai-embed-large-v1` и разреженные BM25-эмбеддинги) в сочетании с хранением в Qdrant обеспечивает гибридный поиск, объединяющий семантическую и лексическую релевантность.

3 Многоуровневый пайплайн ранжирования — гибридный RRF в Qdrant, межзапросное RRF-слияние, профильный буст с power-law decay, расширение по графу пререквизитов и MMR-диверсификация — позволяет формировать контекст, адаптированный к текущему состоянию подготовки конкретного ученика, а не к усредненному запросу.

4 Конвейер генерации с многоступенчатой верификацией (структурная, полнотекстовая, математическая и форматная проверки) снижает вероятность предметных неточностей и обеспечивает соответствие итогового конспекта формату ЕГЭ.

5 Все параметры RAG-пайплайна (веса каналов, константа RRF, бонусы графа, порог mastery, параметр затухания) вынесены в конфигурацию и могут настраиваться через переменные окружения, что обеспечивает воспроизводимость экспериментов и возможность тонкой настройки без изменения кода.

6 Экспериментальные прогоны генерации конспектов выполнялись на основе анонимизированной выгрузки решений учеников с портала «ЕГЭ Прорыв» [42] (выборка из 36 обучающихся; персональные данные удалены в соответствии с требованиями 152-ФЗ); пример результата приведён в приложении.

4 Глава 4. Интеграция с веб-платформой и эксплуатационные характеристики сервиса

В предыдущих главах описаны архитектурные принципы сервиса и реализация RAG-пайплайна генерации конспектов. Эта глава посвящена прикладной интеграции разработанного решения с действующей образовательной платформой ЕГЭПрорыв [42]. Рассматриваются точки взаимодействия с платформой, реализация двух асинхронных потоков обработки данных, гарантии надёжности и аутентификации, а также развёртывание системы в кластере Kubernetes [6]. Поскольку ВКР относится к программному типу, особое внимание уделено эксплуатационным характеристикам (масштабируемость, идемпотентность, отказоустойчивость), которые определяют пригодность сервиса для использования в условиях реальной нагрузки.

4.1 Постановка инженерной задачи интеграции

Платформа ЕГЭПрорыв находится на стадии бета-тестирования и обслуживает реальных учеников [42], поэтому встраиваемый сервис должен соответствовать ряду промышленных ограничений. Во-первых, генерация конспекта средствами LLM является длительной операцией: на используемой модели среднее время одного запроса составляет от 7 до 90 секунд, а в RAG-пайплайне присутствуют дополнительные шаги верификации. Синхронное ожидание такого ответа в HTTP-запросе пользователя недопустимо. Следовательно, генерация должна выполняться асинхронно, с уведомлением платформы через *callback* (HTTP-запрос нашего сервиса в обратную сторону к заранее оговорённому адресу платформы; далее этот термин используется как заимствование без перевода).

Во-вторых, платформа является *системой записи* (system of record): пользовательские данные — профили, история ответов, журналы прогресса — хранятся на стороне платформы. Разрабатываемый сервис должен оставаться *stateless* (без хранения состояния по пользователю между запросами): не дублировать пользовательский контур, не хранить персональные сведения вне жизненного цикла одной операции и не претендовать на роль источника правды о решениях ученика. Такое разделение упрощает требования к защите персональных данных (152-ФЗ), а также снимает с сервиса задачу синхронизации состояния с платформой.

В-третьих, сервис должен быть устойчив к транзиентным сбоям сети, временной недоступности платформы и неоднократной доставке одинаковых событий. Из этих требований вытекает необходимость обеспечения *идемпотентности* операций, корректной обработки

повторных доставок Kafka, а также классификации ошибок на постоянные и временные с соответствующими стратегиями повторов. Без этих свойств всплеск нагрузки или короткое сетевое замешательство приводили бы либо к потере диагностических меток, либо к дублированию конспектов в базе платформы.

Наконец, сервис должен горизонтально масштабироваться. Перед экзаменом число запросов на платформе возрастает; LLM-обработка дороже остальной части пайплайна; векторный поиск по базе знаний нагружается неравномерно. Эффективное решение требует разделения функциональных ролей по разным контейнерам и независимого масштабирования каждой роли. Долгие LLM-операции выносятся в *воркеры* (фоновые процессы-обработчики, не имеющие собственного HTTP-интерфейса и обрабатывающие задания из очереди).

Таким образом, инженерная задача состоит в том, чтобы оформить сервис как набор слабо связанных сервисов, согласованных с платформой через контрактные точки взаимодействия, обладающих свойствами идемпотентности, отказоустойчивости и горизонтальной масштабируемости.

4.2 Контракты взаимодействия с платформой

Интеграция оформлена как явный контракт между двумя сторонами: платформой и разрабатываемым сервисом. Контракт зафиксирован в трёх независимых спецификациях, расположенных в каталоге `app/spec/` репозитория проекта.

1 `generator.openapi.yaml` — эндпоинты, реализуемые нашей стороной: `POST /jobs`, `GET /jobs/{id}`, `GET /healthz`, `GET /readyz`.

2 `platform.openapi.yaml` — эндпоинты, реализуемые платформой и получающие callback'и от нас: `POST /v1/personal-conspect` и `POST/v1/submissions/{id}/diagnostic-tags`.

3 `kafka_submission_event.schema.yaml` — схема событий в топике `ege.submissions.v1`, продьюсером которого является платформа, а консьюмером — наш сервис.

Контракт разделён на три файла, чтобы каждая сторона имела самодостаточное описание *того, что обязана реализовать она сама*; перекрёстные ссылки между файлами отсутствуют. Использование машиночитаемых OpenAPI/JSON Schema форматов даёт обеим сторонам общий формальный источник правды и при необходимости позволяет автоматически генерировать клиентский код, валидаторы и серверные заглушки; в текущем MVP такая генерация не используется, но возможность сохраняется.

Концептуально интеграция образует две асинхронные цепочки обработки данных, далее именуемые *потоком 1* (фоновая диагностика отправленных ответов) и *потоком 2* (генерация персонализированного конспекта по требованию). Поток 1 инициируется платформой через Apache Kafka [22] на каждом отправленном ответе ученика; поток 2 инициируется явным HTTP-запросом со стороны платформы, когда пользователь нажимает в интерфейсе кнопку получения конспекта. Каждый поток рассмотрен ниже отдельно.

4.3 Поток 1: фоновая диагностика ответов учеников

Поток 1 реализует автоматическое извлечение диагностических меток (tags) из каждого неправильного ответа ученика. Меткой считается категориальный ярлык типичной ошибки, например `lost_parentheses_or_sign` или `units_mismatch`. Поток функционирует в фоне и не блокирует пользовательский сценарий: ученик получает немедленный отклик о правильности ответа от самой платформы, а диагностика накапливается асинхронно и используется впоследствии при генерации конспекта. Общая схема потока приведена на Рисунке 4.1.

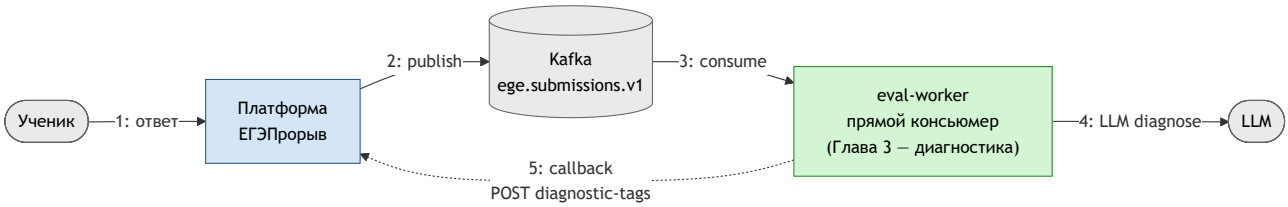

Рисунок 4.1 – Поток 1 – фоновая диагностика ответов учеников. Платформа публикует событие в Kafka на каждом ответе; eval-worker асинхронно извлекает событие, выполняет LLM-диагностику и возвращает метки callback'ом. Зелёные блоки – наша сторона, синие – платформа, серые – внешние компоненты.

Источник событий. Платформа выступает продьюсером в Apache Kafka [22]: при сохранении нового ответа ученика она публикует событие `submission.created` в топик `ege.submissions.v1` [33]. Событие содержит идентификатор отправки, идентификаторы ученика и задания, текст задачи, ответ ученика и эталонный ответ. Apache Kafka обеспечивает доставку *не менее одного раза* (at-least-once) и сохраняет сообщения в логе с упорядочением по партициям, что позволяет надёжно воспроизводить поток событий при сбоях консьюмеров [22].

Стоит отметить, что между сохранением ответа в БД платформы и публикацией события в Kafka возможно «окно потери»: если процесс на стороне платформы упадёт между этими двумя операциями, ответ окажется записан в её базе, но в Kafka не попадёт, и наш сервис диагностику по нему не выполнит. Транзакционная запись в БД и Kafka одной тран-

закцией (transactional outbox) не реализована ни на нашей, ни на платформенной стороне. Этот компромисс согласован с командой платформы: для MVP допустимы единичные потери диагностических меток, поскольку они не нарушают пользовательский сценарий (ученик всё равно видит правильность ответа от самой платформы) и со временем компенсируются на уровне накопленной истории. При выходе сервиса в эксплуатацию рекомендуется добавить transactional outbox на стороне платформы, что устранит окно потери.

Архитектурное решение. Изначально рассматривался вариант *queue-on-database*: введение промежуточного контура `ingest-consumer` → таблица очереди `submission_evaluations` → `eval-worker`. Такая прослойка давала бы дополнительный буфер для дедупликации, кэширование одинаковых ответов между учениками и упрощённую наблюдаемость через SQL-запросы к таблице. Однако в условиях единственного консьюмера, идемпотентного по входу LLM-этапа и относительно небольшой и быстрой обработки одного события (порядка секунды на современных моделях) такая прослойка не оправдывала операционных издержек: дополнительные две операции записи в БД на каждое событие, отдельный процесс `ingest-consumer` с собственной точкой отказа, необходимость очистки таблицы. В финальной архитектуре `eval-worker` реализован как *прямой консьюмер Kafka*: он самостоятельно читает события из топика, выполняет диагностику и отправляет результат на платформу. Тем самым из системы удалена промежуточная таблица очереди и связанные с ней операции записи и уборки. Идемпотентность сохраняется на стороне платформы за счёт заголовка `Idempotency-Key` callback'а, что подробнее обсуждается в подразделе 4.5.

В отличие от потока 1, в потоке 2 (генерация конспекта, подраздел 4.4) мы оставили очередь в БД именно из-за тех свойств, которые здесь оказались избыточными: задание генерации длится десятки секунд и требует трекинга промежуточного состояния, по нему ведётся опрос статуса со стороны платформы, а ошибки генерации требуют видимого учёта попыток.

Реализация консьюмера. Воркер реализован на основе библиотеки `aiokafka` [2] с явным управлением смещениями (*offsets* — позиции консьюмера в логе Kafka, отмечающие, какие сообщения уже обработаны; опция `enable_auto_commit=false` отключает автоматический коммит смещений, передавая управление коду воркера). В каждой итерации цикла из топика читается пачка (batch) до восьми сообщений (`getmany(max_records=8)`); сообщения проходят валидацию по схеме события, после чего асинхронно (`asyncio.gather`) для каждого выполняется диагностика по пайплайну Главы 3 и callback `POST/v1/submissions/{id}/diagnostic-tags`. Лишь после того, как обработка пачки завершилась без транзиентных

ошибок, выполняется коммит смещения. Если хотя бы один callback завершился временной неудачей, коммит пропускается, и Kafka на следующей итерации повторно доставит всю пачку. Корректные ответы при таком повторе обрабатываются повторно, но платформа дедуплицирует их по `Idempotency-Key` равному `submission_id`, поэтому повторная обработка безопасна. LLM-этап вынесен в пул потоков (`run_in_executor`), чтобы синхронные вызовы клиента модели не блокировали event-loop консьюмера.

Горизонтальное масштабирование. Несколько экземпляров `eval-worker` объединены в одну группу консьюмеров `ege-eval-worker`; Kafka распределяет партиции топика между активными экземплярами. Верхняя граница числа полезных реплик равна числу партиций топика; превышение этой границы оставляет лишние экземпляры без партиций. В нашей конфигурации топик `ege.submissions.v1` содержит 1 партицию (для бета-этапа достаточно), однако перепартиционирование поддерживается без изменения кода консьюмера.

4.4 Поток 2: генерация персонализированного конспекта по запросу

Поток 2 реализует синхронно-инициируемую асинхронную операцию: платформа просит сгенерировать конспект и затем получает результат либо callback'ом нашего сервиса по адресу, который платформа сама указала в исходном запросе, либо периодическим опросом по идентификатору задания. Это паттерн *accept-202 then callback*, характерный для длительных операций в HTTP-API. Общая схема потока приведена на Рисунке 4.2.

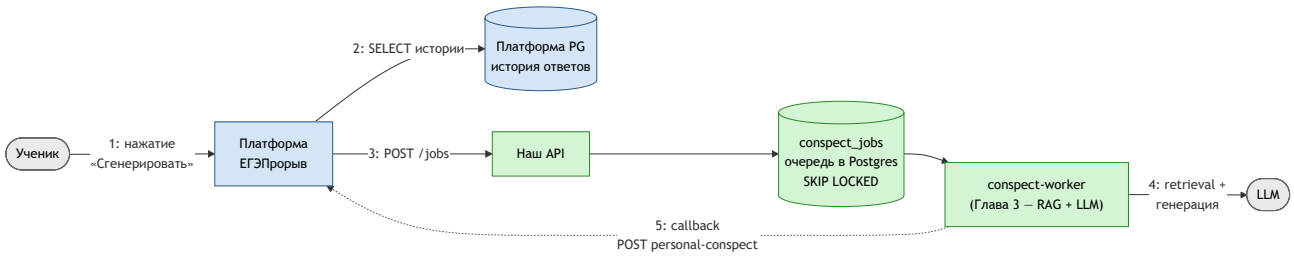

Рисунок 4.2 – Поток 2 – генерация персонализированного конспекта по запросу. Платформа выбирает историю ответов и передаёт её в нашем API; задание помещается в очередь `conspect_jobs` в Postgres, откуда conspect-worker конкурентно захватывает его через `SELECT FOR UPDATE SKIP LOCKED`, выполняет RAG-пайплайн Главы 3 и возвращает конспект callback'ом. Цветовая кодировка как на Рисунке 4.1.

Точка входа. Эндпоинт `POST /jobs` принимает тело запроса с идентификатором задания (`job_id`, выбираемым на стороне платформы), идентификатором ученика, опциональным описанием callback'а и массивом исторических отправок ученика. Сервер возвращает ответ `202 Accepted` с идентификатором задания и его текущим статусом. Если задание с

таким `job_id` уже существует (повторная доставка одного и того же запроса от платформы), возвращается `200 OK` с тем же телом ответа (`job_id`, статус, `accepted_at`), что и в первый раз; новой записи в очередь не создаётся и текущая операция не прерывается. Это обеспечивает идемпотентность точки входа.

Очередь заданий и претензионная семантика. Принятое задание сохраняется в таблицу `conspect_jobs` в Postgres. Использование таблицы как очереди, а не отдельного брокера сообщений, обусловлено двумя соображениями. Во-первых, поток 2 имеет естественную потребность в *состоянии задания* — очередь, выполнение, успех, ошибка, попытки, результат генерации, — которое естественно ложится на реляционную модель и используется тем же эндпоинтом `GET /jobs/{job_id}` для опроса статуса. Во-вторых, число заданий сравнительно невелико (десятки в час, не миллионы в секунду), что не требует производительности журнала Kafka.

Воркеры конкурентно «захватывают» очередное задание с помощью паттерна `SELECT ... FOR UPDATE SKIP LOCKED` [29], реализуемого с PostgreSQL начиная с версии 9.5. Этот паттерн позволяет нескольким экземплярам `conspect-worker` одновременно опрашивать очередь, не блокируя друг друга: строка, на которую уже наложен `FOR UPDATE` другим транзакционным контекстом, просто пропускается следующим запросом. Перевод задания в статус `running` выполняется в той же транзакции, что устраняет гонку между захватом и пометкой.

Жизненный цикл задания. После захвата воркер реконструирует временный профиль ученика по присланной истории отправок (модуль `profile_materializer`), запускает RAG-пайплайн Главы 3, после чего записывает результат в поле `result` (JSONB) и переводит задание в статус `done`. Затем выполняется callback `POST /v1/personal-conspect` по адресу из поля `callback.url`; в случае успеха выставляется `posted_at`. Если callback завершился постоянной ошибкой (4xx), задание переходит в статус `failed`, поскольку повторные попытки не имеют смысла. При временной ошибке (5xx, тайм-аут, сетевой сбой) задание остаётся в статусе `done`, увеличивается счётчик `retry_count`; результат сохранён в БД и может быть повторно доставлен отдельным *sweep-процессом* (фоновое периодическое задание, которое выбирает строки в статусе `done` с непустым `retry_count` и невыставленным `posted_at` и пытается доставить callback ещё раз). В текущем MVP такой sweep не реализован: транзиентный сбой обратной доставки оставляет конспект в БД, и платформа получает его при следующем опросе `GET /jobs/{job_id}`; реализация sweep оставлена на этап выхода из бета-тестирования, когда требования к гарантиям доставки будут зафиксированы.

Полное состояние и опрос. Эндпоинт `GET /jobs/{job_id}` возвращает текущее со-

стояние задания и — при готовности — сгенерированный конспект. Это позволяет платформе использовать обе модели потребления результата: либо получать его *push*-уведомлением через callback, либо извлекать *pull*-моделью при запросе пользователя. На практике callback требует от платформы поднимать публично-достижимый эндпоинт, тогда как опрос проще в развёртывании. Pull-модель особенно дёшева в поддержке на нашей стороне: она не вводит дополнительной инфраструктуры (только тот же `GET /jobs/{id}`, читающий уже существующее состояние из таблицы `conspect_jobs`), не требует от платформы выставлять входящий сетевой порт и устойчива к кратковременной недоступности любой стороны. По этой причине pull-модель используется и нашим внутренним диагностическим модулем (`admin/`, см. подраздел 4.6): он периодически опрашивает `GET /jobs/{id}`, чтобы отображать состояние очередей и результаты сгенерированных конспектов в панели наблюдаемости. Разрабатываемый сервис поддерживает оба сценария потребления одновременно.

Очистка таблицы заданий. Поскольку конспект уже доставлен платформе callback'ом или опросом, в нашей таблице `conspect_jobs` он держится только для целей наблюдаемости и повторной доставки при сбое. В MVP очистка не автоматизирована: ожидается единичные тысячи строк в день, что для Postgres несущественно. На будущее предполагается фоновое задание (`cron` или Kubernetes `CronJob`), удаляющее записи в статусах `done` и `failed` старше N дней (например, 30 суток); правильное значение N зависит от регламента наблюдаемости и согласовывается с командой платформы при выходе из бета-этапа.

4.5 Семантика надёжности и аутентификации

Идемпотентность. Все операции, изменяющие состояние, идемпотентны по явному ключу. `POST /jobs` использует поле тела `job_id` как первичный ключ таблицы `conspect_jobs` в сочетании с PostgreSQL-конструкцией `INSERT ... ON CONFLICT DO NOTHING`: повторное обращение возвращает уже сохранённое задание без побочных эффектов. Обратные вызовы к платформе включают заголовок `Idempotency-Key`, который равен `submission_id` для потока 1 и `job_id` для потока 2 [17]. Платформа обязана дедуплицировать вызовы по этому ключу. Такой подход следует общепринятой практике HTTP-API при работе с at-least-once доставкой и необходим, поскольку `eval-worker` может повторно отправить пачку после транзиентного сбоя, а `conspect-worker` — повторить вызов после временной ошибки.

Политика повторов. Все исходящие вызовы платформы проходят через клиент `PlatformClient`, реализованный на основе `httpx` [8] и `tenacity` [11]. Стратегия повторов

соответствует контракту: на ответы 5xx, тайм-ауты и сетевые ошибки производится до трёх попыток с экспоненциальным увеличением паузы (минимум 2 секунды, максимум 30 секунд). На 4xx ответы (за исключением 408 — тайм-аут запроса — и 429 — ограничение частоты) повторов не делается: такая ошибка считается *постоянной* и оборачивается в исключение `PermanentDeliveryError`. Если все три попытки исчерпаны, бросается `TransientDeliveryError`. Воркеры используют тип исключения как сигнал: `eval-worker` пропускает коммит Kafka и заново отрабатывает пачку, `conspect-worker` увеличивает `retry_count` задания, не меняя состояние `done`. Тем самым внешняя ненадёжность платформы изолируется от внутреннего состояния системы.

Классификация ошибок. Деление на постоянные и временные ошибки — ключевое архитектурное решение, отличающее систему от простой «повторять при любой ошибке» модели. Бесконечные повторы при постоянной ошибке (например, неправильно сформированный JSON) приводили бы к зависанию пайплайна; одноразовая попытка при временном сбое (например, перезапуск платформы) приводила бы к потере данных. Явная классификация на уровне HTTP-кода обеспечивает корректное поведение в обоих случаях.

Аутентификация. Для машинной (service-to-service) аутентификации выбран механизм *подписанного JWT с разделяемым секретом по алгоритму HS256* [18]. Это *минимально достаточное* решение: оно не требует дополнительной инфраструктуры (выделенного сервера авторизации, JWKS-эндпоинта, управления PKI), допускает ротацию через выкатку нового секрета, при этом поддерживается стандартной HTTP-схемой `Authorization: Bearer`. Альтернативные варианты — статический API-ключ, JWT с RS256, OAuth2 client credentials, mTLS — рассматривались, но отклонены как либо менее безопасные, либо избыточно сложные для текущей стадии развития платформы. Выбранный механизм оставляет возможность миграции на более тяжёлые схемы по мере роста требований.

4.6 Развёртывание в Kubernetes и среда разработки

Контейнеризация. Все компоненты сервиса собираются в один контейнерный образ на основе Python 3.12. Зависимости управляются через `uv` [3] с фиксацией точных версий в `uv.lock`; внутри контейнера используется команда `uv sync -frozen -no-dev`, что обеспечивает воспроизводимость окружения между разработкой и продом. Один и тот же образ запускается с разными командами для каждой роли, что упрощает диспетчерское и CI-обеспечение.

Топология кластера. Развёртывание описано манифестами в каталоге `k8s/` (Табли-

ца 4.1) и применяется к выделенному namespace `ege` в кластере Kubernetes [6]. В таблице для каждой роли указан целевой механизм масштабирования: HPA (Horizontal Pod Autoscaler — штатный механизм Kubernetes, изменяющий число реплик деплоймента в зависимости от целевого показателя, в нашем случае загрузки CPU) либо вертикальное масштабирование одиночного StatefulSet'а для зависимостей с локальным состоянием.

Таблица 4.1 – Состав развёртывания в Kubernetes

Манифест	Роль	Реплики	Масштабирование
`api.yaml`	Обработчик `POST /jobs`	2	HPA, до 6
`eval-worker.yaml`	Консьюмер Kafka (поток 1)	2	HPA, по партициям
`conspect-worker.yaml`	Воркер очереди БД (поток 2)	1	HPA, до 4
`admin.yaml`	Симулятор платформы для разработки	1	—
`deps-postgres.yaml`	Postgres для очереди и метаданных	1 (StatefulSet)	вертикально
`deps-redpanda.yaml`	Брокер совместимый с Kafka	1 (StatefulSet)	вертикально
`deps-qdrant.yaml`	Векторная БД базы знаний	1 (StatefulSet)	вертикально
`migrations-job.yaml`	Однократный Job миграций Alembic	1	—

Стратегии масштабирования. Для каждого компонента выбран тип масштабирования, соответствующий его модели потребления нагрузки. `ege-api` stateless и свободно масштабируется по числу входящих HTTP-запросов. `eval-worker` ограничен сверху числом партиций Kafka: дополнительные реплики выше этого предела не получают партиций и простаивают. `conspect-worker` масштабируется свободно благодаря безконфликтной очереди `SKIP LOCKED` в БД. Лимиты ресурсов для воркеров заданы с учётом особенностей пайплайна: `eval-worker` тратит память в основном на буферы Kafka и LLM-вызов, `conspect-worker` — на загруженные модели эмбеддингов и Qdrant-клиент; `terminationGracePeriodSeconds` для conspect-worker увеличен до 60 секунд, чтобы выполняющийся LLM-вызов успевал завершиться при плановом перезапуске пода.

Зависимости и хранение. Postgres, Redpanda и Qdrant развёрнуты в том же namespace как одиночные StatefulSet'ы с persistent volume claims. В качестве брокера сообщений выбран Redpanda [33] — бинарно-совместимая с Apache Kafka реализация на C++, которая использует тот же сетевой протокол и тот же формат сообщений, но не

требует JVM, ZooKeeper или KRaft-кластера для запуска. Для бета-этапа это даёт меньший потребляемый объём памяти и более короткое время старта пода; в случае миграции на полноценный Kafka-кластер код сервиса (использующий клиент `aiokafka`) меняться не должен. Такая схема является достаточной для бета-тестирования; для промышленной эксплуатации рекомендуется заменить эти зависимости на управляемые сервисы (например, Amazon RDS для PostgreSQL, Amazon MSK — Managed Streaming for Apache Kafka, Qdrant Cloud) без изменения кода — параметры подключения вынесены в ConfigMap и Secret. Kubernetes-манифесты и конфигурации не содержат секретов непосредственно; в производственной среде предполагается интеграция с External Secrets Operator и централизованным хранилищем секретов команды ЕГЭПрорыв (например, HashiCorp Vault или облачным менеджером секретов).

Модуль admin: симулятор и панель наблюдаемости. Поскольку реальная платформа находится вне зоны ответственности разработчика, для отладки потоков 1 и 2 разработан модуль `admin/`, совмещающий две роли. Во-первых, он выступает *симулятором платформы*: публикует события `submission.created` в Kafka при отправке ответа через UI, принимает callback'и `POST/v1/submissions/{id}/diagnostic-tags` и `POST /v1/personal-conspect`, а также проксирует `POST /jobs` к нашему API, подставляя действительный сервисный токен вместо плейсхолдера, использованного в браузере. Во-вторых, он служит *панелью наблюдаемости* для самого MVP: на одной странице сведены состояние очередей диагностики и конспектов, задержка обработки, пропускная способность за минуту, P50/P95 длительности и Kafka-лаг группы `ege-eval-worker`. Это особенно полезно до интеграции с полноценной системой мониторинга (Prometheus, Grafana) и позволяет отлаживать сервис в кластере без дополнительной инфраструктуры. В кластере модуль развёрнут отдельным деплойментом и доступен через `kubectl port-forward`; в производственный контур платформы при выходе из MVP он не передаётся, его роль наблюдаемости заменяется выкаткой metrics-эндпоинтов и систем логирования (Prometheus, Grafana, ELK или аналогичных), уже действующих в инфраструктуре команды ЕГЭПрорыв.

4.7 Выводы по главе

1 Интеграция с платформой ЕГЭПрорыв оформлена как явный контракт из трёх независимых спецификаций (две OpenAPI-схемы и одна JSON-схема Kafka-события), что позволяет каждой стороне разрабатывать свою часть автономно и автоматически генерировать клиентский код.

2 Поток 1 (фоновая диагностика ответов) реализован как прямой консьюмер Apache Kafka без промежуточной таблицы очереди, что упрощает систему по сравнению с альтернативой queue-on-database при сохранении идемпотентности за счёт заголовка `Idempotency-Key` и классификации ошибок на постоянные и временные.

3 Поток 2 (генерация конспекта по запросу) использует таблицу `conspect_jobs` как конкурентно-безопасную очередь с захватом строк через `SELECT FOR UPDATE SKIP LOCKED`, что позволяет масштабировать обработку по числу воркеров без дополнительной синхронизации.

4 Семантика надёжности обеспечивается явной классификацией ошибок на постоянные и временные, идемпотентностью всех изменяющих состояние операций по бизнес-ключу и единой стратегией повторов с экспоненциальным увеличением паузы.

5 Сервис развёрнут в Kubernetes как набор деплойментов с независимым горизонтальным масштабированием (HPA по CPU); ограничения и стратегии масштабирования каждого компонента согласованы с его моделью нагрузки.

6 Для локальной разработки и сквозного тестирования контракта интеграции реализован симулятор платформы (модуль `admin/`), полностью повторяющий обратные стороны контракта и предоставляющий панель мониторинга реального времени; в производственный контур платформы симулятор не входит.

Список литературы

1. Ghodai Abdelrahman, Qing Wang и Bernardo Nunes. “Knowledge Tracing: A Survey”. В: *ACM Computing Surveys* 55.11 (2023), с. 1—37. DOI: 10.1145/3569576.

2. aio-libs. *aiokafka: asyncio client for Apache Kafka*. Дата обращения: 01.05.2026. 2024. URL: https://github.com/aio-libs/aiokafka.

3. Astral. *uv: An extremely fast Python package and project manager*. Дата обращения: 01.05.2026. 2024. URL: https://github.com/astral-sh/uv.

4. Ryan S. J. d. Baker. “Data Mining for Education”. В: *International Encyclopedia of Education*. Под ред. Barry McGaw, Penelope Peterson и Eva Baker. 3rd. Elsevier, 2010.

5. Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever и Dario Amodei. “Language Models are Few-Shot Learners”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 33. 2020, с. 1877—1901.

6. Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer и John Wilkes. “Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade”. В: *ACM Queue*. Т. 14. 1. 2016.

7. Jaime Carbonell и Jade Goldstein. “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries”. В: *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1998, с. 335—336. DOI: 10.1145/290941.291025.

8. Tom Christie. *HTTPX: A next-generation HTTP client for Python*. Дата обращения: 01.05.2026. 2024. URL: https://www.python-httpx.org/.

9. Albert T. Corbett и John R. Anderson. “Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge”. В: *User Modeling and User-Adapted Interaction* 4.4 (1994), с. 253—278. DOI: 10.1007/BF01099821.

10. Gordon V. Cormack, Charles L. A. Clarke и Stefan Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. В: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, с. 758—759. DOI: 10.1145/1571941.1572114.

11. Julien Danjou. *Tenacity: General-purpose retrying library for Python*. Дата обращения: 01.05.2026. 2024. URL: https://github.com/jd/tenacity.

12. Jacob Devlin, Ming-Wei Chang, Kenton Lee и Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. В: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*. 2019, с. 4171—4186.

13. Yixuan Ding, Jingheng Feng, Haochen Guo, Minjie Xu, Xinai Lu и Jiaxin Zhang. “Retrieval-Augmented Generation for Educational Application: A Systematic Survey”. В: *Computers and Education: Artificial Intelligence* 8 (2025), с. 100378. DOI: 10.1016/j.caeai.2025.100378.

14. Chevy van Dorresteijn, Dina Fajardo-Tovar, Natalie Pareja Roblin, Frank Cornelissen, Monique Meij, Joke Voogt и Monique Volman. “What Factors Contribute to Effective Online Higher Education? A Meta-Review”. В: *Technology, Knowledge and Learning* 30 (2025), с. 171—202. DOI: 10.1007/s10758-024-09750-5.

15. FastAPI. *FastAPI Documentation*. Англ. Дата обращения: 07.05.2026. 2026. URL: https://fastapi.tiangolo.com/.

16. Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang и Haofen Wang. “Retrieval-Augmented Generation for Large Language Models: A Survey”. В: *arXiv preprint arXiv:2312.10997* (2024).

17. Jayadeba Jena и Sanjay Dalal. *The Idempotency-Key HTTP Header Field*. Internet-Draft IETF HTTPAPI WG. Дата обращения: 01.05.2026. 2024. URL: https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/.

18. Michael B. Jones, John Bradley и Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519. Дата обращения: 01.05.2026. 2015. URL: https://datatracker.ietf.org/doc/html/rfc7519.

19. Enkelejda Kasneci, Kathrin Sessler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, Jochen Weller,

Jochen Kuhn и Gjergji Kasneci. “ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education”. В: *Learning and Individual Differences* 103 (2023), с. 102274. DOI: 10.1016/j.lindif.2023.102274.

20. Khan Academy. *Khanmigo: AI-powered learning guide*. Англ. Дата обращения: 30.03.2026. 2025. URL: https://www.khanacademy.org/khanmigo.

21. Ean Teng Khor и Mutthulakshmi K. “A Systematic Review of the Role of Learning Analytics in Supporting Personalized Learning”. В: *Education Sciences* 14.1 (2024), с. 51. DOI: 10.3390/educsci14010051.

22. Jay Kreps, Neha Narkhede и Jun Rao. “Kafka: A Distributed Messaging System for Log Processing”. В: *Proceedings of the NetDB*. 2011.

23. LangChain. *LangGraph*. Англ. Дата обращения: 30.03.2026. 2024. URL: https://blog.langchain.com/langgraph/.

24. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel и Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 33. 2020, с. 9459—9474.

25. MAXIMUM. *Курсы подготовки к ЕГЭ: онлайн-обучение для 11 класса*. Дата обращения: 30.03.2026. 2026. URL: https://maximumtest.ru/ege.

26. mixedbread.ai. *mxbai-embed-large-v1: Open Source Embedding Model*. Дата обращения: 01.04.2026. 2024. URL: https://huggingface.co/mixedbread-ai/mxbai-embed-large-v1.

27. Maciej Pankiewicz и Ryan S. Baker. “Generating In-Context, Personalized Feedback for Intelligent Tutors with Large Language Models”. В: *International Journal of Artificial Intelligence in Education* (2025). DOI: 10.1007/s40593-025-00505-6.

28. Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas и Jascha Sohl-Dickstein. “Deep Knowledge Tracing”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 28. 2015.

29. PostgreSQL Global Development Group. *SELECT: Locking clauses, SKIP LOCKED option*. Документация PostgreSQL. Дата обращения: 01.05.2026. 2024. URL: https://www.postgresql.org/docs/current/sql-select.html#SQL-FOR-UPDATE-SHARE.

30. Qdrant. *FastEmbed Documentation*. Англ. Дата обращения: 07.05.2026. 2026. URL: https://qdrant.github.io/fastembed/.

31. Qdrant. *Qdrant Documentation*. Англ. Дата обращения: 31.03.2026. 2026. URL: https://qdrant.tech/documentation/overview/.

32. Quizlet. *Introducing Q-Chat: The first AI tutor built with OpenAI's API*. Англ. Дата обращения: 30.03.2026. 2025. URL: https://quizlet.com/blog/introducing-q-chat.

33. Redpanda Data. *Redpanda: A Kafka-compatible streaming data platform*. Дата обращения: 01.05.2026. 2024. URL: https://redpanda.com.

34. Stephen Robertson и Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". В: *Foundations and Trends in Information Retrieval* 3.4 (2009), с. 333—389. DOI: 10.1561/1500000019.

35. Cristóbal Romero и Sebastián Ventura. "Educational Data Mining and Learning Analytics: An Updated Survey". В: *WIREs Data Mining and Knowledge Discovery* 10.3 (2020), e1355. DOI: 10.1002/widm.1355.

36. Yuhong Shi, Kun Yu, YiFei Dong и Fang Chen. "Large Language Models in Education: A Systematic Review of Empirical Applications, Benefits, and Challenges". В: *Computers and Education: Artificial Intelligence* 10 (2025), с. 100529. DOI: 10.1016/j.caeai.2025.100529.

37. Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave и Guillaume Lample. "LLaMA: Open and Efficient Foundation Language Models". В: *arXiv preprint arXiv:2302.13971* (2023).

38. Ang Wang, Lin Chen и Yuhan Wang. "TutorLLM: Customizing Learning Recommendations with Knowledge Tracing and Retrieval-Augmented Generation". В: *arXiv preprint arXiv:2502.15709* (2025).

39. Lixiang Yan, Lele Sha, Linxuan Zhao, Yuheng Li, Roberto Martinez-Maldonado, Guanliang Chen, Xinyu Li, Yueqiao Jin и Dragan Gašević. "Practical and Ethical Challenges of Large Language Models in Education: A Systematic Scoping Review". В: *British Journal of Educational Technology* 55.1 (2024), с. 90—112. DOI: 10.1111/bjet.13370.

40. Роман Игоревич Агарев. *Оптимизация образовательных траекторий с применением машинного обучения в онлайн-курсах*. Отчёт об исследовательском проекте. Науч. рук. О. В. Тарасов. НИУ ВШЭ, Факультет компьютерных наук, 2025.

41. ЕГЭLand. *Онлайн-школа подготовки к ЕГЭ и ОГЭ ЕГЭLand*. Дата обращения: 30.03.2026. 2026. URL: https://el-ed.ru/.

42. ЕГЭПрорыв. *ЕГЭПрорыв — онлайн-школа подготовки к ЕГЭ и ОГЭ*. Дата обращения: 30.03.2026. 2026. URL: https://exambreakout.ru/.

43. Олег Васильевич Тарасов и Никита Алексеевич Птицын. “Разработка системы адаптивного обучения учащихся старшей школы”. Направление 27.04.07 Наукоёмкие технологии и экономика инноваций. Магистерская диссертация. Московский физико-технический институт (национальный исследовательский университет), 2025.

44. Умскул. *Умскул — онлайн-школа подготовки к ЕГЭ и ОГЭ*. Дата обращения: 30.03.2026. 2026. URL: https://umschool.net/.

45. Яндекс. *Подготовка к ЕГЭ по математике (Яндекс.Образование)*. Дата обращения: 30.03.2026. 2026. URL: https://education.yandex.ru/ege/go/math.

A Пример сгенерированного конспекта

Ниже приведён фрагмент конспекта, сформированного RAG-сервисом для одного из анонимных профилей ученика (идентификатор `archive_u5`) на основе выборки решений с портала ЕГЭПрорыв. Исходные данные о попытках решения задач получены из деперсонализированной выгрузки (36 учеников; удалены ФИО, адреса электронной почты и иные идентификаторы). Полный текст конспекта в формате Markdown сохранён в репозитории разработки как часть архива экспериментальных прогонов.

Введение (фрагмент)

Давайте разберём два важных задания первой части ЕГЭ. Задание №10 — прикладная математика (подстановка в формулу), а №12 — исследование функции с помощью производной. В обоих случаях ответом должно быть целое число или конечная десятичная дробь.

Что важно запомнить (фрагмент)

1 **Производная логарифма:** $(\ln x)' = \frac{1}{x}$**.** Логарифм растёт всё медленнее по мере увеличения x. Если в №12 видишь функцию с ln, помни про ОДЗ: аргумент логарифма всегда > 0.

2 **Производная частного:** $\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$. В формуле производной частного в числителе всегда стоит минус перед слагаемым fg'.

3 **Правило «догона» (преследование с форой):** $t = \frac{S}{v_2 - v_1}$ — время догона при разнице скоростей.

Типичные ошибки (фрагмент)

1 **Игнорирование ограничений (ОДЗ) в текстовых задачах.** При решении квадратного уравнения в №10 могут получиться два корня; скорость, время и длина не могут быть отрицательными — выбирай физически осмысленный корень.

2 **Забытый минус в производной частного.** Путать знак с производной произведения нельзя: в числителе всегда $f'g - fg'$.

Разбор примера: задание №12 (фрагмент)

Условие. Найдите наименьшее значение функции $f(x) = x^2 - 8x + 19$ на отрезке $[1; 5]$.

Решение.

1 Производная: $f'(x) = 2x - 8$.

2 Критическая точка: $2x - 8 = 0 \Rightarrow x = 4$.

3 Проверка: $4 \in [1; 5]$.

4 Значения: $f(1) = 12$, $f(5) = 4$, $f(4) = 3$.

5 Наименьшее из $\{12; 4; 3\}$ равно 3.

Ответ: 3.

Чеклист перед ответом (фрагмент)

- В ответе нет букв, корней, π и обыкновенных дробей (если этого требует формат задания).
- В задаче №12 вычислены значения функции на концах отрезка и в критических точках внутри отрезка.
- Единицы измерения в текстовой задаче №10 приведены к одной системе.